



REVERSE-ENGINEERED ENGINEERING HANDBOOK

# SpeakX AI

The complete engineering & technical-interview handbook for **GD-IMPROVEMENT-APP** — an AI-assisted English speaking, group-discussion, debate and interview coach.

React 18

TypeScript 5

Vite 5

Recharts

Web Speech API

localStorage

Client-only SPA

Repository: [RRNauhwar/GD-IMPROVEMENT-APP](https://github.com/RRNauhwar/GD-IMPROVEMENT-APP)

Prepared for interviews at Google · Microsoft · Amazon · Atlassian · Adobe · Salesforce · Oracle · startups

Every statement below is grounded in the actual source code. Absent features are explicitly marked

**NOT PRESENT** .



# Table of Contents

---

## 1. Executive Summary

Problem, users, pitches (30s / 2m / 5m), limitations, roadmap

---

## 2. Complete Folder Structure

Every file, why it exists, blast radius if removed

---

## 3. Complete Architecture

High/low-level, frontend, "backend", data, auth, AI, API, deployment

---

## 4. End-to-End Request Flow

From page load to rendered coaching report

---

## 5. System Design & Scaling

1 - 1,000,000 users, CAP, bottlenecks, trade-offs

---

## 6. Frontend Deep Dive

Routing, state, hooks, rendering, performance

---

## 7. Backend Deep Dive

What exists vs. what is missing

---

## 8. Data / "Database" Deep Dive

localStorage schema, ER diagram, integrity

---

## 9. API Documentation

Web Speech API + optional OpenAI call

---

## 10. Authentication & Authorization

None — analysis and correct design

---

## 11. AI Module

Prompt flow, tokens, latency, cost, failure modes

---

## 12. Security Audit

API key exposure, XSS, prompt injection, CORS, fixes

---

## 13. Performance Analysis

Bundle, renders, memory, complexity

---

## 14. Code Review

SOLID, DRY, KISS, anti-patterns, refactors

---

## 15. Design Patterns

Patterns actually used vs. absent

---

## 16. Libraries

Every dependency, rationale, alternatives

---

## 17. Deployment

Build, hosting, env, CI/CD (missing)

---

## 18. Complete Interview Questions

Categorized, leveled, with model answers & follow-ups

---

## 19. Live Coding Questions

Derived from this codebase, with solutions

---

## 20. Resume Verification

Claims, exaggerations, stronger wording

---

## 21. Mock Interview

Full escalating transcript

---

## 22. Cheat Sheet & Top 100 Questions

Last-minute revision

---

Scores (architecture, scalability, security, maintainability, code quality, interview-readiness) appear at the end of Section 14 and in the Cheat Sheet.

## 1

# Executive Summary

**SpeakX AI** (repository [GD-IMPROVEMENT-APP](#)) is a browser-based coaching application that listens to a person speaking on a discussion/debate/interview topic and returns a structured, multi-dimensional *coaching report*: scores across 13 speaking dimensions, grammar corrections, fluency metrics (words-per-minute, filler words), speech-structure detection, personalized drills, gamification (XP, levels, streaks, badges), and a progress dashboard that charts improvement across sessions.

## 1.1 What problem does it solve?

Improving spoken English, group-discussion (GD) and interview performance normally requires a human coach who is expensive, not always available, and inconsistent. SpeakX AI provides an always-available, private, *objective-first* practice loop: pick a topic → speak against a timer → receive an immediate, measurable report → practice the auto-generated "next challenge" that targets your weakest dimension.

**The central engineering idea.** The product deliberately separates *objective, deterministic analysis* (computed locally in [src/lib/analysis/engine.ts](#) — WPM, filler counts, vocabulary diversity, rule-based grammar, structure detection) from an *optional, subjective LLM narrative* ([src/lib/llm.ts](#)). The app is fully functional and honest with **zero** external dependencies at runtime; the LLM only layers qualitative prose on top.

## 1.2 Target users

| Segment                                           | Need                                          | How the app serves it (in code)                                                                                            |
|---------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Students preparing for placement GDs / interviews | Repeatable practice + measurable feedback     | 5 modes in <a href="#">SetupScreen.tsx</a> ; <a href="#">interviewReadiness()</a> scores Big-Tech/MBA/Govt/Public-Speaking |
| Non-native English speakers                       | Grammar + pronunciation + vocabulary upgrades | <a href="#">analyzeGrammar()</a> , <a href="#">analyzePronunciation()</a> , <a href="#">WEAK_WORDS</a> upgrades            |
| Professionals prepping for public speaking        | Confidence, pacing, structure                 | Timer + 10s warning in <a href="#">SpeakingScreen.tsx</a> ; <a href="#">analyzeStructure()</a> , fear-signal detection     |
| Self-learners who want a habit loop               | Motivation + streaks + progress trend         | <a href="#">gamification.ts</a> (XP/levels/streaks/badges), <a href="#">Dashboard.tsx</a> trend charts                     |

## 1.3 Business use case & why it exists

- **Positioning:** a "practice-and-measure" tool that can be shipped as a free web app, embedded in ed-tech platforms, or upsold with the AI layer (bring-your-own-key today, managed key later).
- **Cost profile:** because scoring is deterministic and client-side, the marginal cost per session is essentially **zero** unless the optional LLM is enabled. This is a genuine differentiator versus fully-LLM competitors.
- **Privacy:** all data (sessions, progress, key) lives in the user's browser [localStorage](#); nothing is transmitted unless the user opts into the OpenAI call.

## 1.4 Current limitations HONEST

- **Pronunciation & pauses are estimated from text, not audio.** [analyzePronunciation\(\)](#) flags long/known-hard words; [longPauses](#) is derived from WPM, not from the audio signal. This is explicitly labelled in the UI ([PronunciationAnalysis.note](#)).

- **Grammar is rule-based (10 regex rules)** in `grammars.ts` — it catches common patterns (e.g. "should of", "he don't") but is not a full parser.
- **Transcription depends on the Web Speech API** (Chrome/Edge). Other browsers fall back to a typed transcript.
- **No accounts / no cloud sync.** Clearing browser storage erases all history.
- **No backend, database, authentication, tests, or CI/CD.** **NOT PRESENT**

## 1.5 Future improvements (roadmap)

1. Real audio analysis (Web Audio API + a speech model) for true pronunciation, pace and pause detection.
2. A thin backend proxy so the LLM key is never in the browser (fixes the key-exposure risk in §12).
3. Accounts + cloud persistence (Postgres/Supabase) for cross-device progress.
4. Automated tests (Vitest + React Testing Library) around the pure `engine.ts` functions.
5. Route-based code-splitting to shrink the ~610 kB bundle (Recharts dominates).

## 1.6 Elevator pitch (30 seconds)

"SpeakX AI is a browser-based speaking coach. You pick a GD/interview topic, speak against a timer, and instantly get a scored coaching report — grammar fixes, filler-word counts, words-per-minute, structure analysis and targeted drills — plus a progress dashboard and gamified streaks. It runs entirely client-side, so it's private and costs nothing per session, with an optional AI layer for richer feedback."

## 1.7 Interview pitch (2 minutes)

"GD-IMPROVEMENT-APP is a React 18 + TypeScript + Vite single-page app that coaches spoken English. The interesting engineering decision is the separation of concerns between *measurement* and *opinion*. All the objective analysis lives in a pure, dependency-free module, `src/lib/analysis/engine.ts` — its `analyzeSpeech()` function takes a transcript plus timing and returns a fully-typed `CoachReport`. It computes WPM, counts single- and multi-word fillers using a `Set` and regex, measures vocabulary diversity via a type-token ratio, runs ten regex grammar rules, and detects speech structure with discourse-marker lexicons. Because it's pure, it's trivially testable and I actually verified it with a smoke test.

The UI is a small screen state-machine in `App.tsx` — no router library — that flows `setup` → `speaking` → `report` → `dashboard`. Speech capture wraps the browser Web Speech API in a custom hook, `useSpeechRecognition`, which handles Chrome's periodic auto-stop by restarting recognition. Persistence is `localStorage` behind a tiny repository in `storage.ts`. There's an optional LLM enhancement that calls OpenAI directly from the browser using the user's own key, and it degrades gracefully — if the call fails, the deterministic report is still shown. I can walk through any of these, and I'm also happy to talk about what I'd add for production: a key-proxying backend, accounts, and tests."

## 1.8 Interview pitch (5 minutes)

The 5-minute version expands the 2-minute pitch with four "why" decisions and their trade-offs:

| Decision                                 | What I did                                                                                       | Why / trade-off                                                                                     |
|------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Deterministic engine first, LLM optional | All scores from <code>engine.ts</code> ; LLM only adds prose via <code>applyEnhancement()</code> | Zero-cost, offline, testable, honest. Trade-off: heuristics are coarser than an LLM's judgement.    |
| Client-only, localStorage                | No backend; <code>storage.ts</code> repository                                                   | Ships instantly, private, \$0 infra. Trade-off: no cross-device sync, data lost if storage cleared. |
| Screen state-machine, no router          | <code>type Screen</code> union + <code>useState</code> in <code>App.tsx</code>                   | Tiny footprint for 6 screens. Trade-off: no deep-linking / back-button semantics.                   |

| Decision       | What I did                                                                                           | Why / trade-off                                                                |
|----------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Honest metrics | Pronunciation/pauses labelled "estimated";<br><code>MIN_SPEECH_WORDS</code> guard blocks fake scores | Builds user trust. Trade-off: less "magical" than pretending to measure audio. |

I'd close by naming the biggest risk — the OpenAI key is exposed if a user enables the AI layer, because the browser calls the API directly — and the exact fix: a small server-side proxy holding the key, which also unlocks rate-limiting and caching. That shows I understand the security boundary I consciously traded away for a zero-backend MVP.

## 2 Complete Folder Structure

The tree below is the actual repository layout (build output and `node_modules` omitted).

```
GD-IMPROVEMENT-APP/
├── index.html          # Vite HTML entry; mounts #root, loads /src/main.tsx
├── package.json        # scripts + deps (react, react-dom, recharts)
├── package-lock.json   # locked dependency graph
├── tsconfig.json       # app TS config (strict, noEmit, bundler resolution)
├── tsconfig.node.json  # TS config for vite.config.ts (composite)
├── vite.config.ts      # Vite + @vitejs/plugin-react, dev server :5173
├── .gitignore          # ignores node_modules, dist, *.tsbuildinfo, emitted config
├── README.md           # feature docs + run instructions + honest caveats
├── public/
│   └── vite.svg        # favicon (gradient "S" logo)
├── scripts/
│   ├── smoke.ts        # manual functional test of analyzeSpeech()
│   └── render-pdf.mjs   # (docs tooling) HTML→PDF via puppeteer
├── docs/
│   └── handbook.html   # (this document, source)
├── src/
│   ├── main.tsx        # React root: createRoot(...).render(<App/>)
│   ├── App.tsx         # screen state-machine + session orchestration
│   ├── index.css       # full design system (light theme)
│   ├── types.ts        # ALL domain types + DURATION_OPTIONS
│   ├── vite-env.d.ts   # Vite client type reference
│   ├── hooks/
│   │   └── useSpeechRecognition.ts # Web Speech API wrapper hook
│   ├── lib/
│   │   ├── topics.ts   # topic generator (40 seeds × 4 frames)
│   │   ├── storage.ts  # localStorage repository (sessions/game/key)
│   │   ├── gamification.ts # XP, levels, streaks, badges, insights
│   │   ├── llm.ts      # optional OpenAI enhancement + SPEAKX_SYSTEM_PROMPT
│   │   └── analysis/
│   │       ├── engine.ts # analyzeSpeech() — the deterministic core
│   │       ├── text.ts   # tokenizing, sentences, syllables, clamp, countBy
│   │       ├── grammar.ts # 10 regex grammar rules GrammarIssue[]
│   │       └── lexicons.ts # fillers, weak/power/advanced words, markers
│   └── components/
│       ├── SetupScreen.tsx # mode/difficulty/category/duration + topic picker
│       ├── SpeakingScreen.tsx # countdown timer + live transcript capture
│       ├── ReportView.tsx  # the 14-section coaching report (+ radar chart)
│       ├── Dashboard.tsx  # progress trend charts + session history
│       ├── Settings.tsx   # optional OpenAI API key entry
│       └── common.tsx     # ScoreBar, ScoreRing, Section, Stat helpers
```

### 2.1 Why each file exists & blast radius if removed

| File                      | Responsibility                                                         | If removed...                             |
|---------------------------|------------------------------------------------------------------------|-------------------------------------------|
| <code>index.html</code>   | Vite entry; single <code>#root</code> div + module script              | App has no mount point; Vite build fails. |
| <code>src/main.tsx</code> | Bootstraps React 18 <code>createRoot</code> in <code>StrictMode</code> | Nothing renders; blank page.              |



| File                                       | Responsibility                                                                                                                           | If removed...                                                          |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>src/App.tsx</code>                   | Top-level state-machine; owns sessions, gamification, active config; orchestrates <code>handleFinish</code>                              | No navigation/orchestration; app is non-functional.                    |
| <code>src/types.ts</code>                  | Single source of truth for every domain type; <code>DURATION_OPTIONS</code>                                                              | Type errors cascade through the whole codebase.                        |
| <code>src/index.css</code>                 | Entire design system (theme vars, cards, timer, ring, charts)                                                                            | App renders unstyled.                                                  |
| <code>lib/analysis/engine.ts</code>        | <b>The core.</b> <code>analyzeSpeech()</code> + <code>MIN_SPEECH_WORDS</code> guard                                                      | No coaching report can be produced; product loses its reason to exist. |
| <code>lib/analysis/text.ts</code>          | Pure text utilities ( <code>words</code> , <code>sentences</code> , <code>syllables</code> , <code>clamp</code> , <code>countBy</code> ) | Engine + grammar break (shared primitives).                            |
| <code>lib/analysis/grammar.ts</code>       | Rule-based grammar detection                                                                                                             | Grammar section of report is empty; scores shift.                      |
| <code>lib/analysis/lexicons.ts</code>      | Word lists (fillers, markers, weak/power words)                                                                                          | Fluency/structure/vocabulary analysis collapse.                        |
| <code>lib/topics.ts</code>                 | Topic + "next challenge" generation                                                                                                      | No topics to speak on; setup + next-challenge break.                   |
| <code>lib/storage.ts</code>                | localStorage repository (persistence boundary)                                                                                           | No history, gamification, or key persistence.                          |
| <code>lib/gamification.ts</code>           | XP/level/streak/badge + cross-session insights                                                                                           | Report loses XP banner & insights; dashboard still works.              |
| <code>lib/llm.ts</code>                    | Optional OpenAI enhancement + system prompt                                                                                              | Only the optional AI narrative is lost; deterministic app unaffected.  |
| <code>hooks/useSpeechRecognition.ts</code> | Web Speech API lifecycle (start/stop/restart/errors)                                                                                     | Live transcription breaks; only typed fallback would remain.           |
| <code>components/*</code>                  | Presentational screens                                                                                                                   | Corresponding screen disappears.                                       |
| <code>scripts/smoke.ts</code>              | Manual functional check of the engine                                                                                                    | No automated confidence in engine changes (it is not a CI gate).       |

**Layering observation.** The dependency direction is clean and one-way: `components`  $\square$  `lib`  $\square$  `types` , and within `lib/analysis` , `engine`  $\square$  `{grammar, text, lexicons}` . Nothing in `lib/` imports from `components/` . This is why the engine is unit-testable in isolation (as `smoke.ts` proves).

## 3

## Complete Architecture

**Reality check before we draw boxes.** Many "architecture" boxes people expect (API gateway, microservices, database cluster, auth server) *do not exist* in this project. This is a **client-only SPA**. Below, each diagram shows what is real; anything not in the code is labelled **NOT PRESENT**. Inventing them would fail an honest reverse-engineering review.

### 3.1 High-level architecture

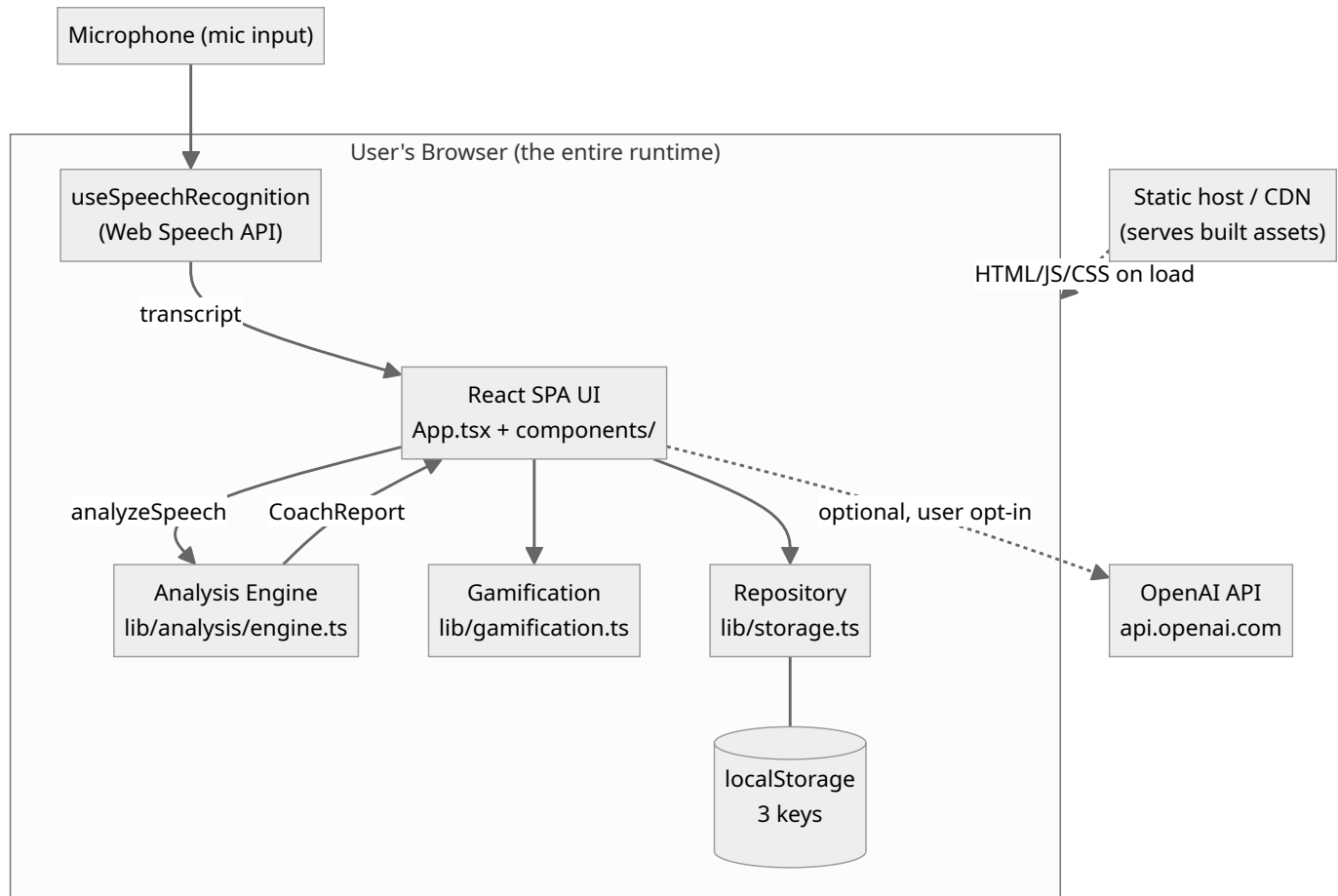


Figure 3.1 — Everything except the optional OpenAI call and static hosting runs inside one browser tab.

### 3.2 Low-level module architecture (component diagram)

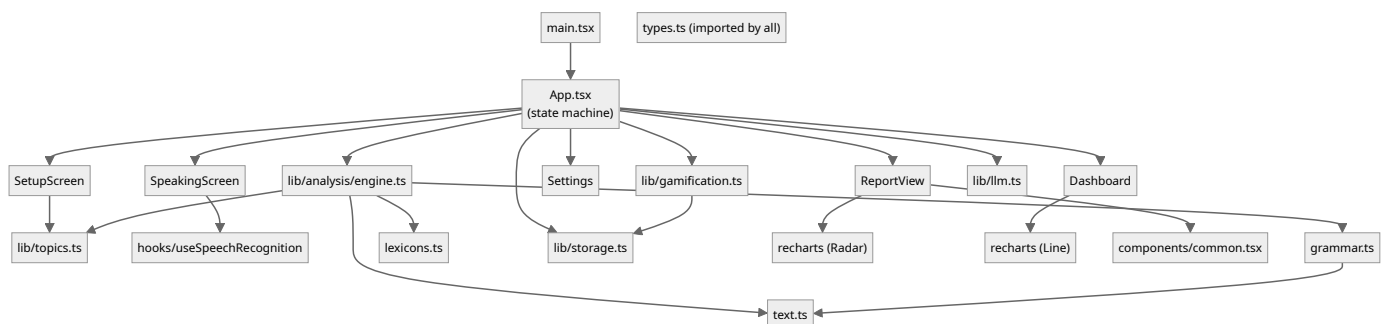


Figure 3.2 — Import graph. Note the one-way flow: components → lib → types.

### 3.3 Frontend architecture

Three layers, no framework beyond React:

| Layer                 | Files                                                                                                                                               | Role                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Presentation          | <code>components/*</code> , <code>common.tsx</code> , <code>index.css</code>                                                                        | Pure rendering; props in, callbacks out. No business logic.    |
| Orchestration / state | <code>App.tsx</code> , <code>hooks/useSpeechRecognition.ts</code>                                                                                   | Owns app state, drives screen transitions, wires domain calls. |
| Domain / logic (pure) | <code>lib/analysis/*</code> , <code>lib/topics.ts</code> , <code>lib/gamification.ts</code> , <code>lib/storage.ts</code> , <code>lib/llm.ts</code> | Framework-agnostic TypeScript. Testable without React.         |

### 3.4 "Backend" architecture **NOT PRESENT**

There is no server, no Node/Express, no serverless functions, no container. The only network egress the code can make is the optional `fetch("https://api.openai.com/...")` in `llm.ts` , executed by the browser. All "backend-like" work (scoring, persistence) happens client-side. §7 covers what a real backend would look like and why it is the #1 production gap.

### 3.5 Data architecture (localStorage)

The "database" is three JSON blobs in `localStorage` , namespaced with a version suffix (see `storage.ts` ):

| Key                                 | Shape                                                           | Writer                                                                             |
|-------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>speakx.sessions.v1</code>     | <code>Session[]</code> (newest first via <code>unshift</code> ) | <code>saveSession</code> , <code>deleteSession</code> , <code>clearSessions</code> |
| <code>speakx.gamification.v1</code> | <code>GamificationState</code>                                  | <code>saveGamification</code> (via <code>awardForSession</code> )                  |
| <code>speakx.openai.key.v1</code>   | <code>string</code> (raw key)                                   | <code>saveApiKey</code>                                                            |

Full ER-style model and integrity discussion in §8.

### 3.6 Authentication & Authorization architecture **NOT PRESENT**

There is no login, no user identity, no JWT/OAuth/session/cookie, and therefore no authorization. The app is single-user-per-browser. This is architecturally coherent for a local tool, but it means "your data" is really "this browser's data." §10 analyses this and sketches the correct design if accounts were added.

### 3.7 AI architecture

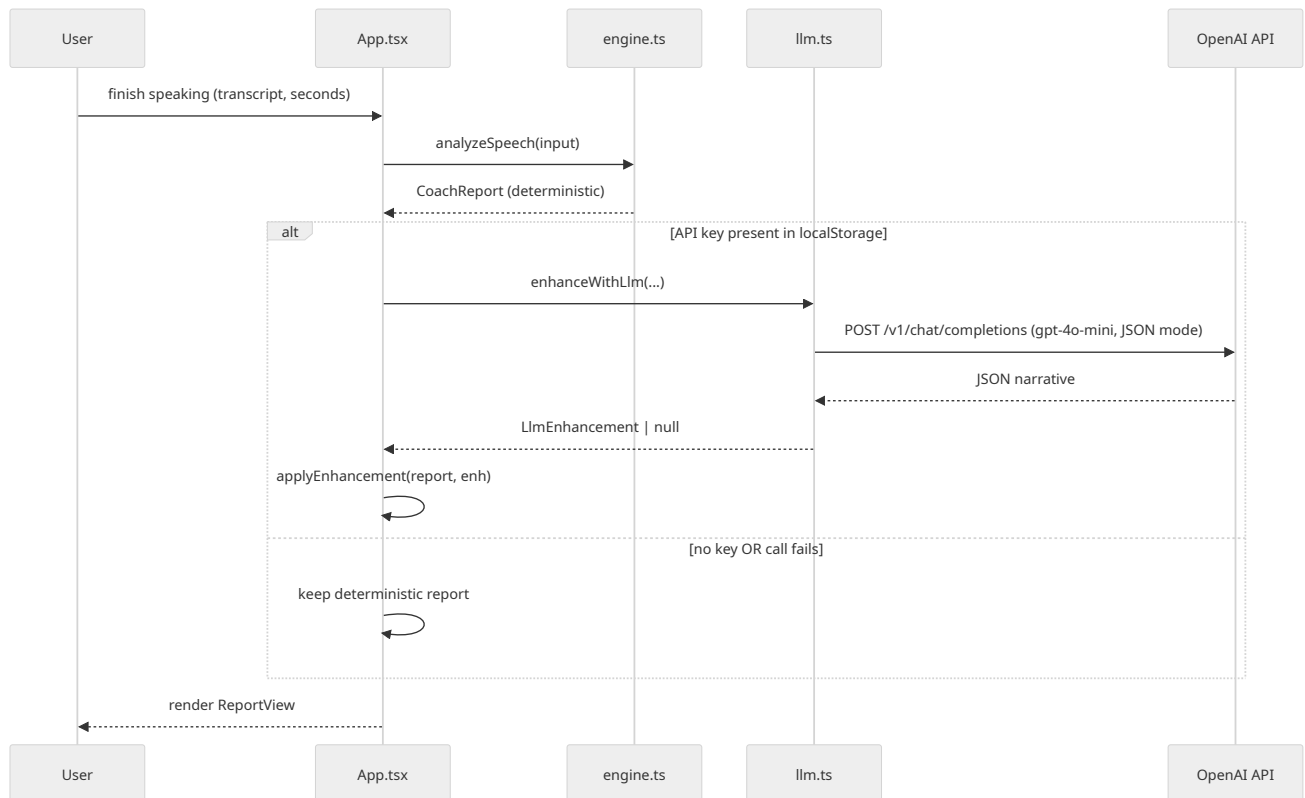


Figure 3.7 — The LLM is strictly an enhancer; the app never depends on it. Detail in §11.

### 3.8 API architecture

The app exposes **no** API of its own. It *consumes* two browser/remote APIs:

- **Web Speech API** ( `SpeechRecognition` / `webkitSpeechRecognition` ) — local, in-browser speech-to-text, wrapped by `useSpeechRecognition` .
- **OpenAI Chat Completions** — optional, called directly from the browser in `llm.ts` .

Full request/response contracts in §9.

### 3.9 Deployment architecture

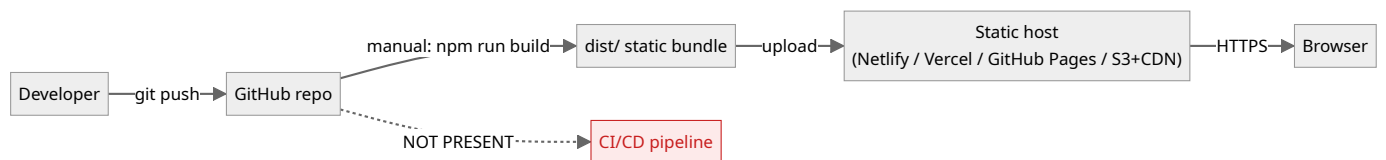


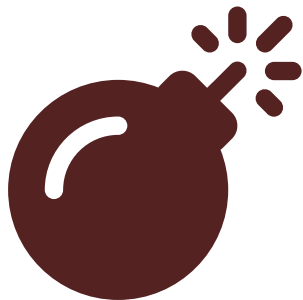
Figure 3.9 — Build is a static bundle; any static host works. No CI/CD is configured (§17).

## 4 End-to-End Request Flow

The prompt's canonical flow assumes a server, login, DB lookup and an API tier. This project has none of those, so below is a *faithful mapping*: for each canonical step I state what actually happens (or mark it **NOT PRESENT**), then give the true flow.

| Canonical step     | Reality in this codebase                                                                                                                                                           |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User opens website | Static host serves <code>index.html</code> + hashed JS/CSS. Browser runs <code>main.tsx</code> □ mounts <code>&lt;App/&gt;</code> .                                                |
| Login              | <b>NOT PRESENT</b> — no auth screen. App opens straight to <code>SetupScreen</code> .                                                                                              |
| Authentication     | <b>NOT PRESENT</b> .                                                                                                                                                               |
| Database lookup    | Client-side: <code>loadSessions()</code> / <code>loadGamification()</code> read from <code>localStorage</code> during <code>useState</code> initialisers in <code>App.tsx</code> . |
| API                | No first-party API. Web Speech API (local) provides transcript; OpenAI (optional) provides narrative.                                                                              |
| Backend processing | Runs in the browser: <code>analyzeSpeech()</code> + <code>awardForSession()</code> + <code>buildInsights()</code> .                                                                |
| AI request         | Optional <code>enhanceWithLlm()</code> □ OpenAI, only if a key exists.                                                                                                             |
| Response           | A <code>CoachReport</code> object (plus <code>XpResult</code> , insights) held in React state.                                                                                     |
| Frontend rendering | <code>ReportView</code> renders the 14 sections; Recharts draws the radar chart.                                                                                                   |

### 4.1 The real end-to-end sequence



## Syntax error in text mermaid version 10.9.1

Figure 4.1 — Every arrow corresponds to a real function call in the source.

### 4.2 Step-by-step narration

- Cold load.** The static host returns `index.html`; Vite's hashed module bundle boots React. `App`'s `useState` lazy initialisers call `loadSessions()` / `loadGamification()` so prior history is available on first paint.
- Setup.** `SetupScreen` lets the user choose mode, difficulty, category, duration; `generateTopic()` produces a framed prompt. `onStart` lifts the config into `App.active` and switches `screen` to `"speaking"`.
- Speaking.** `SpeakingScreen` starts the timer and calls `useSpeechRecognition.start()`. The hook creates a `SpeechRecognition` instance, sets `continuous` + `interimResults`, and accumulates final chunks. A `setInterval` ticks the countdown; at □10s a warning shows; at 0 it auto-finishes.
- Finish & guard.** `onFinish` computes `countSpokenWords`. If below `MIN_SPEECH_WORDS` (12), it routes to the `"nospeech"` screen and does *not* score/save/award — the fix that stops "marks for silence."

5. **Analyse.** Otherwise `analyzeSpeech()` returns a fully-typed `CoachReport`. If a key exists, `enhanceWithLlm()` may add prose; failures fall back silently.
6. **Persist & reward.** `buildInsights()` compares against history; `awardForSession()` updates XP/level/streak/badges and persists; `saveSession()` prepends the session.
7. **Render.** `screen="report"` mounts `ReportView`; Recharts renders the radar; the user can jump to the "next challenge" or the dashboard.

## 5 System Design & Scaling

**Framing.** A client-only static SPA has an unusual scaling profile: the "server" is just a CDN serving immutable files, so raw traffic scales almost for free. The real scaling questions here are (a) the optional OpenAI dependency, (b) client-side storage limits, and (c) bundle delivery. I'll answer the classic sub-topics against *this* reality rather than pretend there's a fleet of app servers.

### 5.1 Concept-by-concept

| Concept                   | Current state                                                                                                                       | What it would take                                                        |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Scalability               | Static assets scale via CDN edge caching; compute is on each user's device                                                          | Nothing for the core; only the AI proxy (if added) needs to scale         |
| Availability              | = CDN availability (typically 99.9%+). No server to fall over.                                                                      | Multi-region CDN; SLA on the future proxy                                 |
| Fault tolerance           | LLM failures are swallowed ( <code>enhanceWithLlm</code> returns <code>null</code> ); <code>safeParse</code> guards corrupt storage | Retry/backoff + circuit breaker on the proxy                              |
| Performance               | Analysis is O(n) over words; renders are cheap; bundle ~610 kB is the cost                                                          | Code-split Recharts; lazy-load Dashboard                                  |
| Caching                   | Browser HTTP cache for hashed assets; localStorage as data cache                                                                    | Service worker (PWA) for offline; CDN cache headers                       |
| Rate limiting             | <b>NOT PRESENT</b> (user's own OpenAI quota applies)                                                                                | Enforce at the proxy per-user                                             |
| Queue system              | <b>NOT PRESENT</b> / not needed                                                                                                     | Only if batch/async AI jobs are introduced                                |
| Load balancing            | Handled by CDN                                                                                                                      | ALB/NLB in front of the proxy fleet                                       |
| Horizontal scaling        | Implicit (every client is its own compute)                                                                                          | Stateless proxy replicas behind LB                                        |
| Vertical scaling          | N/A — no server to size                                                                                                             | N/A                                                                       |
| Microservices vs monolith | Neither — it's a <b>frontend monolith</b> (one SPA)                                                                                 | A single proxy service is enough; microservices would be over-engineering |

### 5.2 CAP theorem

CAP governs distributed data stores. Here, data is **single-node** (one browser's localStorage), so CAP barely applies: there is no partition between replicas because there are no replicas. If cloud sync were added, the honest choice for this domain (coaching history) is **AP with eventual consistency** — a user should always be able to record a session offline (availability) and sync later; a few seconds of staleness across devices is harmless. Strong consistency (CP) is unnecessary because there are no concurrent writers to the same record.

### 5.3 Database selection (if/when added)

| Option                   | Fit for this app                                                                                                                                      |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL (or Supabase) | <b>Best default.</b> Sessions are relational (user $\square$ sessions $\square$ report JSONB). Supabase also gives auth + row-level security cheaply. |

| Option   | Fit for this app                                                                                                       |
|----------|------------------------------------------------------------------------------------------------------------------------|
| MongoDB  | Viable — <code>CoachReport</code> is already a nested document; but relational reporting/aggregation is easier in SQL. |
| Firebase | Fast to ship, good offline sync, but vendor lock-in and awkward complex queries.                                       |
| Redis    | Only as a cache / rate-limit store in front of the proxy, not as the primary store.                                    |

## 5.4 Bottlenecks (today)

1. **Bundle size (~610 kB, Recharts-dominated).** First-load latency on slow networks. Fix: dynamic `import()` for chart-heavy screens.
2. **localStorage ceiling (~5 MB).** Each session stores the full `CoachReport` + transcript. Thousands of sessions could approach the limit; there is no pruning.
3. **OpenAI rate limits & latency** (only when AI enabled) — a per-key concern today; a shared-proxy concern later.
4. **Web Speech API** quality/availability varies by browser and is network-backed in Chrome.

## 5.5 How it supports 100 – 1,000,000 users

| Scale     | Does it hold? | Required changes                                                                                                                    |
|-----------|---------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 100       | ☐ As-is       | None. A single static host is plenty.                                                                                               |
| 1,000     | ☐ As-is       | Put the static bundle behind a CDN; add basic web analytics.                                                                        |
| 10,000    | ☐ Core; ☐ AI  | If AI is on, introduce the key-proxy with per-user rate limiting + response caching.                                                |
| 100,000   | ☐             | Accounts + cloud DB for cross-device history; proxy autoscaling; observability (metrics/logs/traces); code-split bundle.            |
| 1,000,000 | ☐             | Multi-region CDN + DB read replicas; queue LLM calls with backpressure; cost controls & abuse prevention; feature-flagged rollouts. |

**Interview-grade takeaway:** "Because scoring is client-side and deterministic, the app scales to a million users on a CDN with essentially no backend. The only component that needs classic horizontal scaling is the AI proxy I'd add to hide the key — and that's a stateless service, so it scales linearly behind a load balancer with Redis for rate-limiting and caching."



## 6

## Frontend Deep Dive

## 6.1 Routing — a screen state-machine (no router library)

Navigation is a discriminated string union in `App.tsx`, switched with `useState`:

```
type Screen = "home" | "speaking" | "report" | "dashboard" | "settings" | "nospeech";
const [screen, setScreen] = useState<Screen>("home");
```

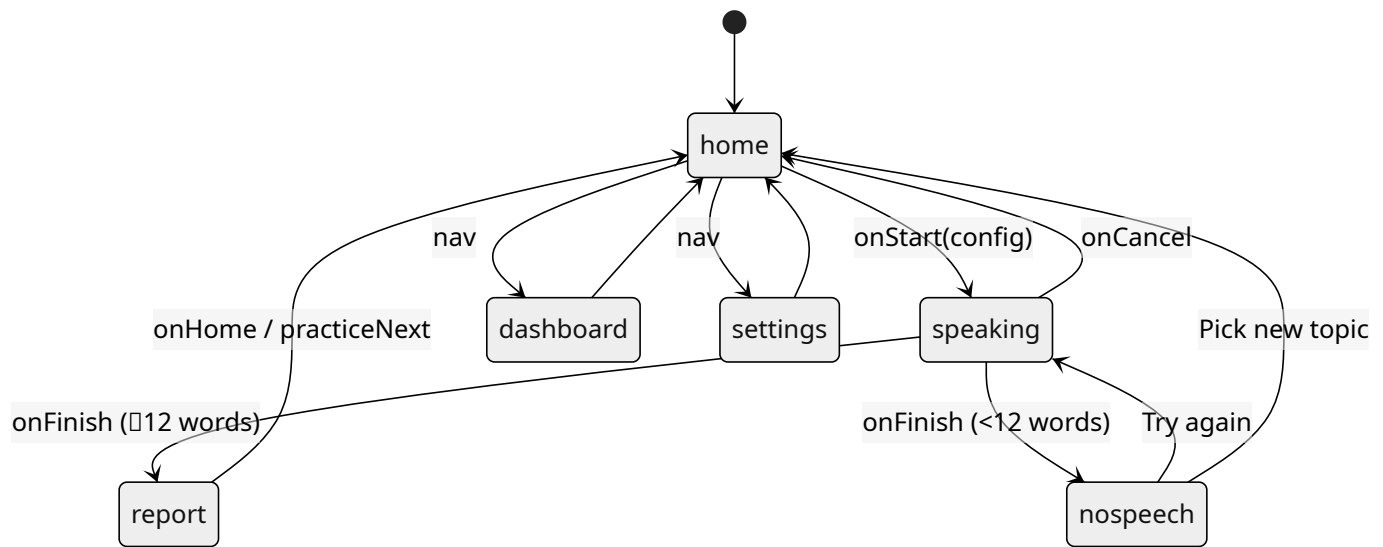


Figure 6.1 — Six states; transitions are plain `setScreen()` calls. No URL involvement.

**Trade-off:** zero dependencies and trivial to reason about, but no deep-linking, no browser back/forward semantics, and refresh always returns to `home`. For a tool this size that's acceptable; at product scale you'd adopt React Router.

## 6.2 State management

All state is local React state — **no Redux, Zustand, MobX, or Context**. `App.tsx` is the single owner of cross-screen state and passes data down via props; children communicate up via callbacks ("lifting state up"). The pieces:

| State                                                                                       | Type                             | Purpose                            |
|---------------------------------------------------------------------------------------------|----------------------------------|------------------------------------|
| <code>screen</code>                                                                         | <code>Screen</code>              | Which view is mounted              |
| <code>sessions</code>                                                                       | <code>Session[]</code>           | History (seeded from localStorage) |
| <code>game</code>                                                                           | <code>GamificationState</code>   | XP/level/streak/badges             |
| <code>active</code>                                                                         | <code>ActiveConfig   null</code> | Config for the in-progress session |
| <code>report</code> , <code>currentSession</code> , <code>insights</code> , <code>xp</code> | various                          | Data for the report screen         |
| <code>analyzing</code> , <code>spokenWords</code> , <code>presetTopic</code>                | bool / number / Topic            | Transient UI flags                 |

## 6.3 Hooks used

| Hook                                        | Where                                                                                                                    | Why                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>useState</code>                       | App, all screens                                                                                                         | Local state, incl. lazy initialisers reading localStorage |
| <code>useMemo</code>                        | App ( <code>prog = levelProgress(game)</code> ), <code>SetupScreen</code> (categories)                                   | Avoid recompute per render                                |
| <code>useEffect</code>                      | <code>useSpeechRecognition</code> (detect support, cleanup), <code>SpeakingScreen</code> (interval cleanup)              | Side effects + teardown                                   |
| <code>useRef</code>                         | <code>useSpeechRecognition</code> (recognizer, "should listen" flag), <code>SpeakingScreen</code> (timer id, start time) | Mutable values that must not trigger re-render            |
| <code>useCallback</code>                    | <code>useSpeechRecognition</code> ( <code>start/stop/reset</code> )                                                      | Stable function identities                                |
| custom<br><code>useSpeechRecognition</code> | <code>SpeakingScreen</code>                                                                                              | Encapsulates Web Speech API lifecycle                     |

## 6.4 The custom hook (concurrency & the Chrome auto-stop problem)

Chrome's `SpeechRecognition` fires `onend` periodically even mid-speech. The hook keeps a `shouldListenRef` and restarts recognition on `onend` until the user stops — a small but real piece of event-loop/async reasoning:

```
rec.onend = () => {
  if (shouldListenRef.current) {
    try { rec.start(); } catch { /* ignore double-start */ }
  } else {
    setListening(false);
  }
};
```

## 6.5 Rendering

Rendering is conditional JSX in `App.tsx` gated by `!analyzing && screen === "..."`. Screens mount/unmount on transition, which naturally resets their internal state (e.g. the timer resets when re-entering `SpeakingScreen`). Lists use stable keys ( `session.id` , indices for static arrays).

## 6.6 API calls, forms & validation

- **API call:** the single `fetch` is in `llm.ts` (§9/§11).
- **Forms:** controlled inputs — the API key field in `Settings.tsx` and the manual-transcript `<textarea>` in `SpeakingScreen.tsx`; chip "selectors" in `SetupScreen` / `Dashboard`.
- **Validation:** the meaningful rule is the `MIN_SPEECH_WORDS` guard (§4). Inputs are otherwise free-text; no schema validation library is used.

## 6.7 Performance, code-splitting, lazy loading, memoization

- **Memoization:** `useMemo` / `useCallback` as tabled above. Components are not wrapped in `React.memo` (not needed at this tree size).
- **Code-splitting / lazy loading:** **NOT PRESENT**. Everything ships in one chunk; Recharts is the largest contributor (§13).

- Renders:** because `App` owns state, a state change re-renders the active screen subtree — acceptable given each screen is small and charts are leaf components.

## 6.8 Error handling (frontend)

| Site                              | Strategy                                                                                                                                                                                    |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useSpeechRecognition</code> | <code>onerror</code> ignores <code>no-speech</code> / <code>aborted</code> ; surfaces others to <code>error</code> state; <code>try/catch</code> around <code>start()/stop()/abort()</code> |
| <code>storage.ts</code>           | <code>safeParse</code> wraps <code>JSON.parse</code> in <code>try/catch</code> □ returns fallback on corruption                                                                             |
| <code>llm.ts</code>               | Whole call in <code>try/catch</code> ; non-OK responses and parse errors return <code>null</code> □ deterministic report shown                                                              |
| <code>main.tsx</code>             | Throws early if <code>#root</code> is missing (fail fast)                                                                                                                                   |

**Gap:** there is no React **Error Boundary**. An unexpected render-time exception would blank the app. Adding one around the screen switch is a cheap resilience win.

## 7

## Backend Deep Dive

**There is no backend.** No routes, controllers, middleware, server-side services, or logging infrastructure exist in the repository. Rather than fabricate them, this section maps each backend concern to where the equivalent responsibility currently lives on the client, then specifies the backend I would build for production.

## 7.1 Backend concerns mapped to the client

| Backend concept  | Client-side equivalent today                                                                                                  |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Routes           | <code>screen</code> union + conditional render in <code>App.tsx</code>                                                        |
| Controllers      | Handlers in <code>App.tsx</code> : <code>startSession</code> , <code>handleFinish</code> , <code>practiceNext</code>          |
| Services         | <code>lib/analysis/engine.ts</code> , <code>lib/gamification.ts</code> , <code>lib/topics.ts</code> , <code>lib/llm.ts</code> |
| Repository / DAO | <code>lib/storage.ts</code> over <code>localStorage</code>                                                                    |
| Middleware       | <b>NOT PRESENT</b> (closest: the <code>MIN_SPEECH_WORDS</code> guard acts like a request validator)                           |
| Validation       | <code>countSpokenWords</code> / <code>hasSufficientSpeech</code> ; <code>safeParse</code>                                     |
| Auth / AuthZ     | <b>NOT PRESENT</b> (§10)                                                                                                      |
| Business logic   | Pure functions in <code>lib/</code>                                                                                           |
| Error handling   | try/catch + null-fallbacks (§6.8)                                                                                             |
| Logging          | <b>NOT PRESENT</b> (no <code>console</code> logging in app code except the standalone <code>smoke.ts</code> )                 |
| Configuration    | Build-time only: <code>vite.config.ts</code> , <code>tsconfig*.json</code> . No runtime env vars.                             |

## 7.2 The backend I would add for production

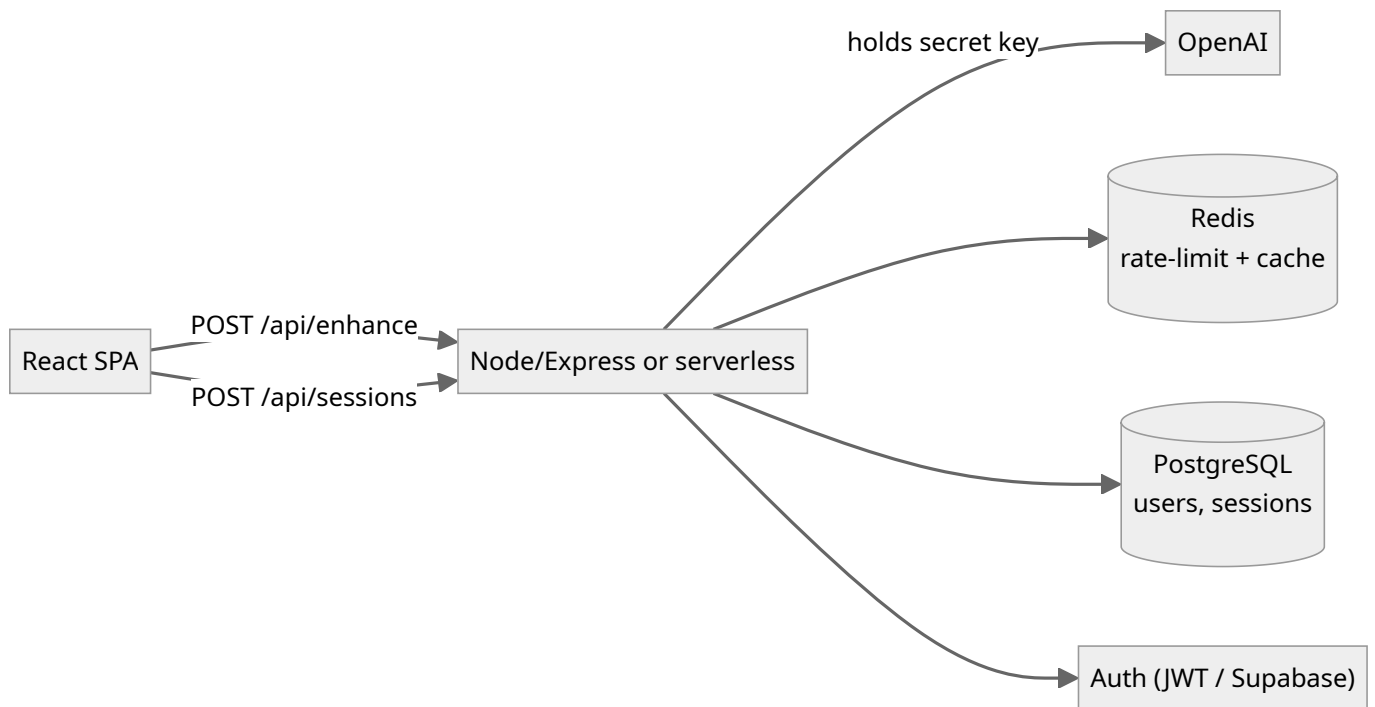


Figure 7.2 — Minimal, non-over-engineered backend: one service, one DB, one cache, one auth provider.

- **/api/enhance** — proxies OpenAI so the key never reaches the browser; enforces per-user rate limits and caches identical transcript requests.
- **/api/sessions** — CRUD for cloud-synced history (replaces localStorage).
- **Auth** — JWT access token + refresh token, or Supabase Auth with row-level security.
- **Logging/observability** — structured logs, metrics, request tracing.

## 8 Data / "Database" Deep Dive

There is no SQL/NoSQL database. The persistent store is `localStorage` with three keys (\$3.5). However, the *data model* is rich and fully typed in `types.ts`, so we can treat the TypeScript interfaces as the schema and draw a real ER diagram of the object graph that is serialised to JSON.

### 8.1 ER diagram (logical model from types.ts)

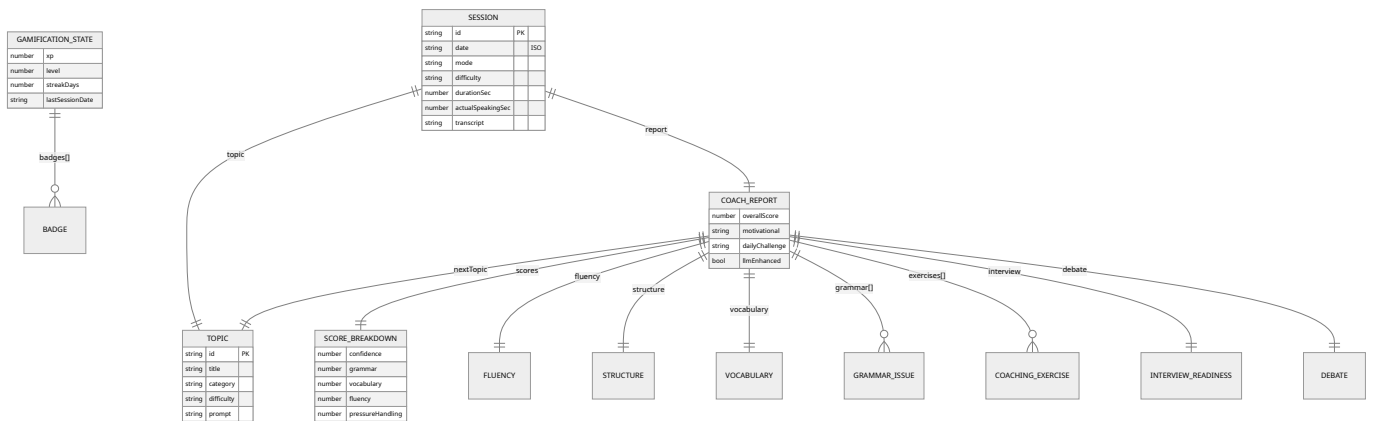


Figure 8.1 — The object graph serialised into `speakx.sessions.v1` and `speakx.gamification.v1`. (Selected fields shown; see `types.ts` for the full set.)

### 8.2 "Tables" (localStorage keys) and their fields

#### Session (element of `speakx.sessions.v1`)

| Field                                                     | Type         | Notes                                          |
|-----------------------------------------------------------|--------------|------------------------------------------------|
| <code>id</code>                                           | string       | Primary key; <code>sess_\${Date.now()}}</code> |
| <code>date</code>                                         | string (ISO) | Used for dashboard ordering & streaks          |
| <code>topic</code>                                        | Topic        | Embedded (denormalised)                        |
| <code>mode</code> , <code>difficulty</code>               | enums        | Session config                                 |
| <code>durationSec</code> / <code>actualSpeakingSec</code> | number       | Target vs. actual; actual drives WPM           |
| <code>transcript</code>                                   | string       | Raw text analysed                              |
| <code>report</code>                                       | CoachReport  | Embedded full report (denormalised)            |

#### GamificationState (`speakx.gamification.v1`)

| Field                        | Type          | Notes                                             |
|------------------------------|---------------|---------------------------------------------------|
| <code>xp</code>              | number        | XP within current level (decremented on level-up) |
| <code>level</code>           | number        | Starts at 1; threshold <code>level*100</code>     |
| <code>streakDays</code>      | number        | +1 if gap==1 day, reset if gap>1                  |
| <code>lastSessionDate</code> | string   null | ISO; basis for streak math                        |

| Field               | Type                  | Notes                                          |
|---------------------|-----------------------|------------------------------------------------|
| <code>badges</code> | <code>string[]</code> | Unlocked badge ids (set-like, dedup on insert) |

## 8.3 Relationships, indexes, queries

- **Relationships** are by *embedding*, not foreign keys: a `Session` embeds its `Topic` and full `CoachReport`. This is classic document-style denormalisation — one read yields everything to render a session.
- **Indexes:** **NOT PRESENT** — `localStorage` has none. "Queries" are just array methods: `filter` (delete), `reverse` (chronological chart), `reduce` (averages in `buildInsights`).
- **Ordering:** newest-first is maintained by `unshift` on write; the dashboard `reverse()` s a copy for oldest→newest trends.

## 8.4 Normalisation, transactions, constraints

| Aspect        | State                                                                                                                                                                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Normalisation | Deliberately <b>denormalised</b> (embedded documents). Fine for read-mostly, single-user data.                                                                                                                         |
| Transactions  | <b>NOT PRESENT</b> . Writes are single <code>setItem</code> calls — atomic per key, but multi-key updates (session + gamification) are <i>not</i> transactional. A crash between the two could desync XP from history. |
| Constraints   | Enforced only by TypeScript types at compile time; no runtime schema validation of what's read back (a hand-edited <code>localStorage</code> blob could violate the types).                                            |

**Optimisation opportunities:** (1) cap stored sessions or drop the transcript for old sessions to respect the ~5 MB limit; (2) add a `schemaVersion` and a migration on load; (3) validate parsed data (e.g. with Zod) before trusting it.

## 9

# API Documentation

The application defines no first-party HTTP API. It integrates two external/browser APIs. Both are documented below in the requested format.

## 9.1 Web Speech API (browser, local)

| Attribute       | Value                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------|
| Interface       | <code>window.SpeechRecognition ?? window.webkitSpeechRecognition</code>                              |
| Wrapper         | <code>useSpeechRecognition(lang = "en-US")</code>                                                    |
| Config          | <code>continuous = true</code> , <code>interimResults = true</code> , <code>lang = "en-US"</code>    |
| Events consumed | <code>onresult</code> (interim+final), <code>onerror</code> , <code>onend</code> (auto-restart)      |
| Auth            | None (browser mic permission prompt)                                                                 |
| Errors handled  | <code>no-speech</code> / <code>aborted</code> ignored; others → user-facing message + typed fallback |
| Availability    | Chrome/Edge (yes); Firefox/Safari (limited/none) → manual typing                                     |

## 9.2 OpenAI Chat Completions (optional, remote)

| Attribute         | Value                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Method / Endpoint | <code>POST https://api.openai.com/v1/chat/completions</code>                                                                                                   |
| Caller            | <code>enhanceWithLlm()</code> in <code>llm.ts</code> (browser <code>fetch</code> )                                                                             |
| Headers           | <code>Content-Type: application/json</code> , <code>Authorization: Bearer &lt;user key from localStorage&gt;</code>                                            |
| Body              | <code>model: "gpt-4o-mini"</code> , <code>temperature: 0.6</code> , <code>response_format: {type:"json_object"}</code> , <code>messages: [system, user]</code> |
| Auth              | User's own OpenAI key (bring-your-own-key)                                                                                                                     |
| Validation        | Response parsed with <code>JSON.parse</code> ; array fields defensively checked ( <code>Array.isArray</code> )                                                 |
| Error codes       | Any non-OK ( <code>!res.ok</code> ) or thrown error → returns <code>null</code> (graceful fallback)                                                            |

### Example request (body)

```
{
  "model": "gpt-4o-mini",
  "temperature": 0.6,
  "response_format": { "type": "json_object" },
  "messages": [
    { "role": "system", "content": "You are SpeakX AI, ... (SPEAKX_SYSTEM_PROMPT)" },
    { "role": "user", "content": "Topic: ...\nDifficulty: ...\nTranscript: ..." }
  ]
}
```



## Example response (parsed message.content)

```
{
  "detailedFeedback": "You opened with a clear stance and used a concrete example ...",
  "strongIdeas": ["Personal anecdote about sleep", "Acknowledged the counter-view"],
  "weakArguments": ["Claim about focus lacked evidence"],
  "motivational": "You're closer than you think — tighten the ending.",
  "dailyChallenge": "Re-record this topic and cut every 'basically'."
}
```

## Request flow diagram

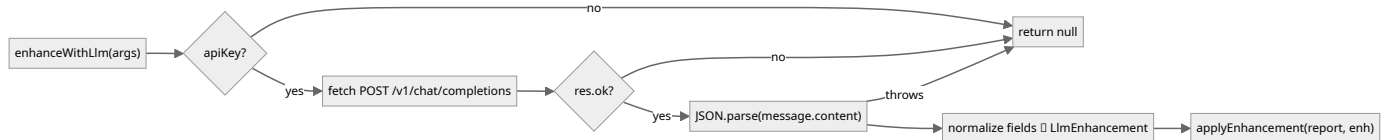


Figure 9.2 — Failure at any step falls back to the deterministic report.

## 10

# Authentication & Authorization

**No authentication or authorization exists.** There is no JWT, OAuth, session, cookie, refresh/access token, password hashing, or role-based access control anywhere in the codebase. The app is single-user-per-browser and stores everything locally.

## 10.1 Why this is (currently) acceptable

Because there is no server and no shared data, there is no identity to authenticate and no resource to authorize access to. Each browser sees only its own `localStorage`. The one secret in play — the optional OpenAI key — is supplied and owned by the user themselves.

## 10.2 What each requested concept *would* be if accounts were added

| Concept            | Design for this app                                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Password hashing   | Never store plaintext; use <code>bcrypt</code> / <code>argon2</code> on a backend. (Better: delegate to Supabase Auth / OAuth and store none.) |
| Access token (JWT) | Short-lived (~15 min), signed, sent as <code>Authorization: Bearer</code> ; carries <code>userId</code> to scope <code>/api/sessions</code> .  |
| Refresh token      | Long-lived, <code>httpOnly</code> + <code>Secure</code> + <code>SameSite</code> cookie; rotated on use.                                        |
| OAuth              | Google/GitHub sign-in — lowest friction for students; no password to manage.                                                                   |
| Sessions / cookies | If server-rendered, <code>httpOnly</code> session cookie; for SPA, token-in-memory + refresh cookie.                                           |
| RBAC               | Minimal: <code>user</code> vs <code>admin</code> . Users can only read/write their own sessions (ownership check / row-level security).        |

## 10.3 Security risks tied to the *absence* of auth

- **No data protection at rest:** anyone with the device can read history via DevTools.
- **Key exposure:** the OpenAI key sits in `localStorage` in plaintext and is attached to a browser request — see §12 for the full analysis and fix.
- **No abuse controls:** without identity there is no per-user rate limiting on the (future) AI proxy.

**Interview soundbite:** "I didn't add auth because there's no server and no shared state — it would be security theatre. The moment I introduce cloud sync, the correct move is OAuth (Google) + short-lived JWT access tokens with an `httpOnly` refresh cookie, and row-level security so a user can only touch their own sessions."

## 11

## AI Module

The AI module is `src/lib/llm.ts`. It is **optional** and **additive**: the deterministic engine always produces the report; the LLM only rewrites the qualitative prose.

## 11.1 How AI is used

- Triggered in `App.handleFinish` only when a key exists *and* the transcript is non-empty.
- `enhanceWithLlm()` sends topic, difficulty, duration, the engine's overall score/WPM/fillers, and the transcript.
- The model returns JSON with five fields; `applyEnhancement()` merges `detailedFeedback`, `strongIdeas`, `weakArguments`, `motivational`, `dailyChallenge` into the report and flips `llmEnhanced = true`.

## 11.2 Prompt flow & prompt engineering

The system prompt (`SPEAKX_SYSTEM_PROMPT`) does three deliberate things:

1. **Role framing**: "world-class communication coach", explicitly "NOT a grammar checker" — steers away from duplicating the deterministic engine.
2. **Division of labour**: tells the model that objective metrics are already computed, so it should add *human, specific* judgement, not recompute numbers.
3. **Structured output**: demands a strict JSON object; combined with the API's `response_format: {type:"json_object"}` (JSON mode), this makes parsing reliable.

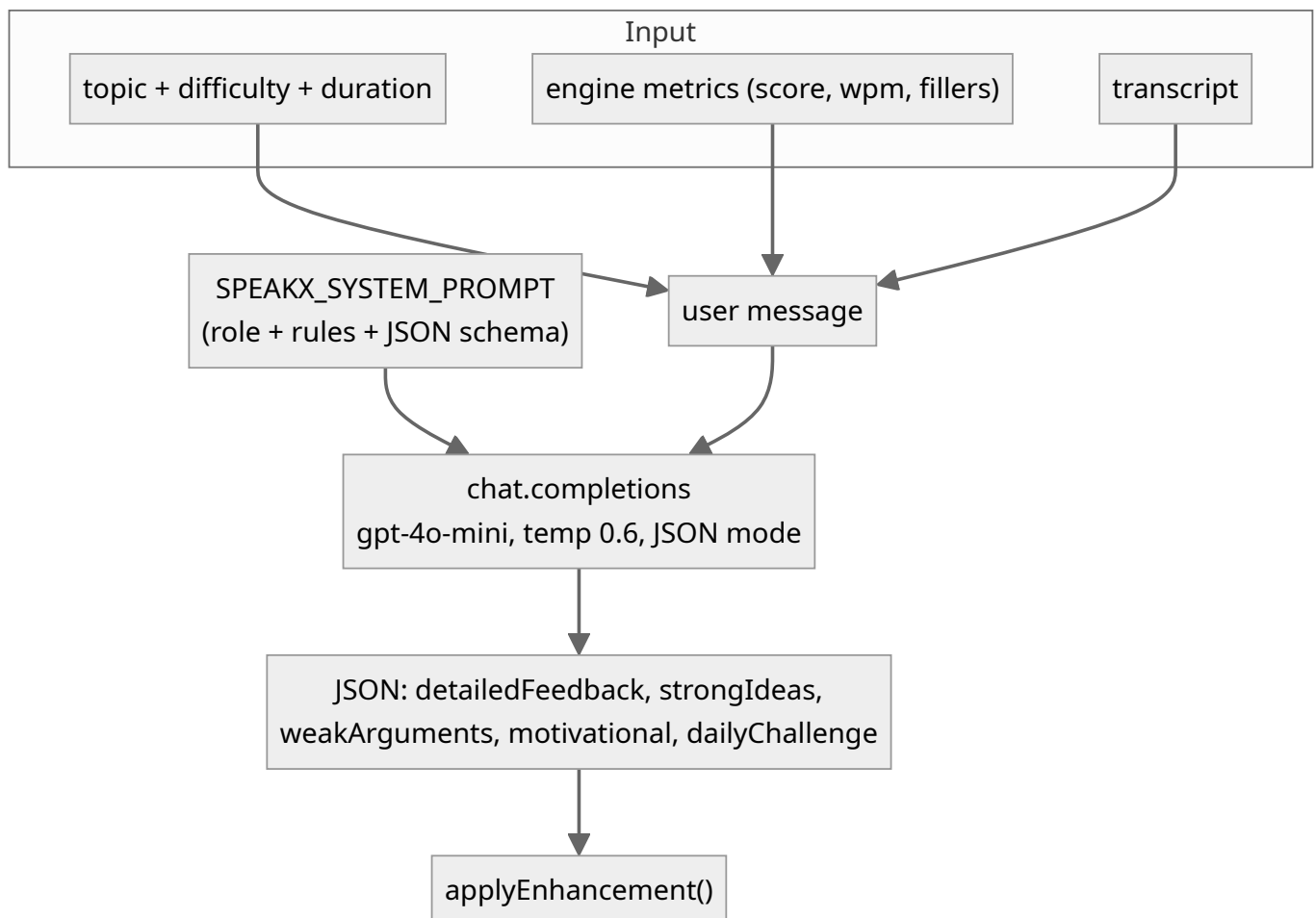


Figure 11.1 — Prompt composition and merge.

## 11.3 Context & memory

**Stateless / single-turn.** Each call is independent — there is no conversation memory, no embedding/RAG, and prior sessions are *not* sent to the model. "Memory" in the product sense (progress over time) is handled deterministically by `buildInsights()` against `localStorage`, not by the LLM.

## 11.4 Token usage, latency, cost

| Factor        | Estimate / control                                                                                                    |
|---------------|-----------------------------------------------------------------------------------------------------------------------|
| Input tokens  | System prompt (~250) + metrics + transcript (a 2-min speech ≈ 300 words ≈ 400 tokens). Typically < ~800 input tokens. |
| Output tokens | Small JSON, ~150–300 tokens.                                                                                          |
| Model         | <code>gpt-4o-mini</code> — chosen for low cost/latency; overridable via <code>args.model</code> .                     |
| Latency       | Typically ~1–3 s; the UI shows an "Analyzing..." state and the deterministic report is ready regardless.              |
| Cost          | Fractions of a cent per session on <code>gpt-4o-mini</code> ; borne by the user's own key today.                      |

## 11.5 Failure cases & handling

| Failure                  | Behaviour                                                                      |
|--------------------------|--------------------------------------------------------------------------------|
| No key                   | Skip entirely; deterministic report shown                                      |
| HTTP error (401/429/5xx) | <code>!res.ok</code> → return <code>null</code> → deterministic report         |
| Malformed JSON           | <code>JSON.parse</code> throws → caught → <code>null</code>                    |
| Partial JSON             | Field-level defaults ( <code>?? ""</code> , <code>Array.isArray</code> guards) |
| Network/CORS failure     | Caught by try/catch → <code>null</code>                                        |

## 11.6 Optimisation & cost optimisation (future)

- **Proxy + cache:** identical transcripts → cached narrative (also hides the key).
- **Debounce/opt-in:** only call when the user requests "deeper feedback", not every session.
- **Prompt trimming:** cap transcript length sent; summarise very long speeches.
- **Model tiering:** mini by default, escalate only on request.

**Prompt-injection note:** the user's transcript is untrusted text placed in the user message. It could contain "ignore previous instructions...". Because the output is constrained to a JSON schema and is only shown back to the same user (never used to take actions), blast radius is low — but §12 still recommends output validation and treating the transcript as data, not instructions.

## 12 Security Audit

Findings are grounded in the actual code. Severity reflects real-world impact for a client-only app.

| # | Finding                                               | Severity | Location                                         |
|---|-------------------------------------------------------|----------|--------------------------------------------------|
| 1 | OpenAI API key exposed in the browser                 | High     | llm.ts , storage.ts , Settings.tsx               |
| 2 | Key stored in <code>localStorage</code> in plaintext  | Medium   | storage.ts ( <code>speakx.openai.key.v1</code> ) |
| 3 | Prompt injection via transcript                       | Low-Med  | llm.ts                                           |
| 4 | No Content-Security-Policy / security headers         | Medium   | index.html / host config                         |
| 5 | No Error Boundary (availability, not confidentiality) | Low      | App.tsx                                          |
| 6 | Untrusted localStorage trusted at read time           | Low      | storage.ts                                       |

### 12.1 Detailed findings & fixes

#### Finding 1 — API key exposure HIGH

When AI is enabled, the browser calls OpenAI directly with `Authorization: Bearer <key>`. The key is therefore present in client memory and visible in the Network tab / any injected script. The code itself acknowledges this (Settings warns the user), but the architecture cannot fully mitigate it.

**Fix:** introduce a backend proxy ( `/api/enhance` ) that holds the key as a server secret; the browser sends only the transcript. This also enables per-user rate-limiting and response caching. This is the single most important production change.

#### Finding 2 — Plaintext key at rest MEDIUM

`saveApiKey` writes the raw string to `localStorage`, readable by any script on the origin (so it compounds with any XSS).

**Fix:** once a proxy exists, don't store the provider key on the client at all; store only a session token for your own backend.

#### Finding 3 — Prompt injection LOW-MED

The transcript is untrusted and embedded in the user message. Blast radius is limited (JSON-mode output shown only to the same user; the app never executes model output). **Fix:** keep JSON mode, validate the parsed object against the expected shape (already partially done), and never feed model output into anything executable.

#### Finding 4 — Missing security headers MEDIUM

No CSP, `X-Content-Type-Options`, `Referrer-Policy`, etc. For a static SPA these are set at the host/CDN. **Fix:** add a strict CSP (allow `self` + `api.openai.com` for `connect-src`), `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`.

#### Findings 5 & 6 — Resilience LOW

Add a React Error Boundary; validate parsed localStorage with a schema (e.g. Zod) and migrate or discard invalid blobs instead of trusting them.

## 12.2 Threats that are *not* applicable

| Threat                | Status | Why                                                                                                               |
|-----------------------|--------|-------------------------------------------------------------------------------------------------------------------|
| SQL Injection         | N/A    | No database, no SQL.                                                                                              |
| CSRF                  | N/A    | No cookies/sessions; no state-changing server endpoints.                                                          |
| Broken access control | N/A    | No multi-user resources to control.                                                                               |
| Server secret leakage | N/A*   | No server; *the client key issue is Finding 1.                                                                    |
| Stored XSS            | Low    | React escapes text by default; no <code>dangerouslySetInnerHTML</code> ; user text is rendered as text, not HTML. |
| CORS misconfig        | N/A    | No first-party API to misconfigure; OpenAI sets its own CORS.                                                     |

**XSS specifically:** a grep of the codebase shows no `dangerouslySetInnerHTML`, no `eval`, and no injection of user text into HTML. React's default escaping makes reflected/stored XSS unlikely with the current rendering approach.

## 13 Performance Analysis

### 13.1 Bundle size (measured)

Production build output (from `npm run build`):

| Asset      | Raw      | Gzip     |
|------------|----------|----------|
| index.html | ~0.62 kB | ~0.38 kB |
| CSS        | ~8.2 kB  | ~2.6 kB  |
| JS bundle  | ~610 kB  | ~175 kB  |

**Largest issue:** the single JS chunk exceeds Vite's 500 kB warning, dominated by **Recharts** (which pulls in D3 modules). It is loaded even on the home screen, which never shows a chart.

**Fixes:** (1) `React.lazy` + dynamic `import()` for `ReportView` / `Dashboard` so Recharts loads only when a chart is shown; (2) `manualChunks` to split the vendor; (3) consider a lighter chart lib.

### 13.2 Rendering & re-renders

- State lives in `App`; a change re-renders the active screen subtree only (others are unmounted).
- No obvious wasteful re-render loops. `useMemo` guards `levelProgress` and category derivation.
- Opportunity:** the countdown `setInterval` in `SpeakingScreen` updates `remaining` every second, re-rendering that screen — cheap, but the live transcript also updates frequently; both are isolated to that screen.

### 13.3 Algorithmic complexity of the engine

| Function                                        | Work                                | Complexity                                          |
|-------------------------------------------------|-------------------------------------|-----------------------------------------------------|
| <code>words()</code> / <code>sentences()</code> | Single regex pass                   | $O(n)$ in transcript length                         |
| <code>countBy()</code>                          | One pass into a <code>Map</code>    | $O(w)$ in word count                                |
| <code>analyzeFluency()</code>                   | Word scan + phrase regexes + counts | $O(w)$                                              |
| <code>analyzeGrammar()</code>                   | 10 rules $\times$ sentences         | $O(r \cdot s)$ — $r=10$ constant $\Rightarrow O(s)$ |
| <code>analyzeStructure()</code>                 | Marker <code>includes</code> scans  | $O(m \cdot n)$ — $m$ constant                       |
| <code>score()</code> / <code>overall()</code>   | Fixed arithmetic over 13 dims       | $O(1)$                                              |
| <code>analyzeSpeech()</code> (whole)            | Sum of above                        | <b><math>O(n)</math></b> in transcript size         |

Space is  $O(w)$  for the frequency maps/sets. For any realistic speech (hundreds of words) this is effectively instant — the deterministic report is not a performance concern.

### 13.4 Memory leaks & network

- Timers/recognizers cleaned up:** `SpeakingScreen` clears its interval on unmount; `useSpeechRecognition` aborts recognition in its cleanup effect. No obvious leaks.
- Network:** only static assets (cacheable, immutable hashes) + the optional OpenAI call. Nothing chatty.

- **localStorage growth:** unbounded session history (§8.4) is the main long-term memory concern on the client.

## 13.5 Caching opportunities

- Hashed static assets → long `Cache-Control: immutable` at the CDN.
- Service worker for offline use (PWA) — the deterministic app works fully offline already.
- LLM response cache keyed by transcript hash (at the future proxy).



## 14

## Code Review

## 14.1 What's done well

- **Separation of concerns:** pure domain logic in `lib/`, presentation in `components/`, orchestration in `App.tsx`. Import direction is one-way.
- **Type safety:** a single `types.ts` is the source of truth; `tsconfig` is `strict` with `noUnusedLocals` / `noUnusedParameters`.
- **Pure, testable core:** `analyzeSpeech` and helpers have no side effects (verified by `smoke.ts`).
- **Graceful degradation:** LLM and storage failures never crash the app.
- **Honest UX:** estimated metrics are labelled; the `MIN_SPEECH_WORDS` guard prevents fake scores.
- **Readable naming:** functions read like their intent ( `analyzeFluency` , `detectFearSignals` , `awardForSession` ).

## 14.2 Principles scorecard

| Principle          | Assessment                                                                                                                                                                           |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SRP (SOLID)</b> | Strong in <code>lib/</code> ; <code>App.tsx</code> is a mild "god component" (state + orchestration + all screen JSX).                                                               |
| <b>Open/Closed</b> | Grammar rules and lexicons are data-driven arrays — extend by adding entries, no logic change. Good.                                                                                 |
| <b>DIP</b>         | Partial: <code>gamification.ts</code> imports <code>storage.ts</code> directly (concrete dependency), making it harder to test in isolation. Injecting the store would improve this. |
| <b>DRY</b>         | Mostly dry; score-label maps appear in both <code>engine.ts</code> and <code>ReportView</code> / <code>Dashboard</code> — a shared label map would remove duplication.               |
| <b>KISS</b>        | Excellent — no premature abstraction, no unnecessary libraries.                                                                                                                      |

## 14.3 Anti-patterns / smells

- **God component:** `App.tsx` holds ~10 state slices and all screen rendering. Extracting a `useSession()` hook and a screen router would help.
- **Hidden coupling to global:** `gamification.ts` reads/writes storage internally, so `awardForSession` has a side effect that isn't obvious from its signature.
- **Non-transactional multi-key writes** (§8.4).
- **Dead-ish code:** `expectedWordsPerSentence` / `sentenceCount` in `analyzeFluency` are computed then `void`-discarded — leftover from an earlier approach.
- **Magic numbers:** scoring weights (e.g. `92` , `fillerRate*350` , `idealWpm=130` ) are literals; extracting a tuning-constants object would aid clarity and testing.
- **No tests in CI:** `smoke.ts` is manual only.

## 14.4 Concrete refactoring opportunities

1. Extract `App` orchestration into `useSpeakingSession()` ; keep `App` as a thin router.
2. Introduce a `SCORE_LABELS` shared constant used by engine + views.
3. Make `awardForSession(state, report)` pure and let the caller persist — removes the hidden side effect and improves testability.
4. Move scoring constants into a `tuning.ts` and add unit tests pinning known transcripts to expected score ranges.
5. Add Vitest + React Testing Library; wire into CI.

## 14.5 Overall scores (out of 10)

| Dimension           | Score      | Rationale                                                                                         |
|---------------------|------------|---------------------------------------------------------------------------------------------------|
| Architecture        | <b>7.5</b> | Clean layering & pure core; limited by god-component and no backend boundary.                     |
| Scalability         | <b>7.0</b> | Static/CDN scales freely; localStorage & bundle are the ceilings.                                 |
| Security            | <b>5.5</b> | Good defaults (React escaping, graceful failures) but the client-side key is a real high-sev gap. |
| Maintainability     | <b>7.5</b> | Strong types, readable names; magic numbers & god-component cost a point.                         |
| Code quality        | <b>7.5</b> | Consistent, idiomatic TS/React; minor dead code.                                                  |
| Interview-readiness | <b>9.0</b> | Rich talking points: pure functions, heuristics, trade-offs, honest gaps, clear roadmap.          |

**Weighted overall** □ **7.3/10** as a portfolio/MVP project — and a **9/10 as an interview vehicle**, precisely because it has clear, defensible decisions and honestly-scoped gaps you can discuss.

## 15

## Design Patterns

Only patterns *actually present* are claimed. Absent ones are marked and explained.

| Pattern                         | Present? | Where / why                                                                                                                                                                   |
|---------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository                      | Yes      | <code>storage.ts</code> abstracts the localStorage persistence boundary behind <code>load*/save*</code> functions.                                                            |
| Facade                          | Yes      | <code>analyzeSpeech()</code> is a facade over fluency/vocab/grammar/structure/scoring sub-analyses.                                                                           |
| Strategy (data-driven)          | Yes      | <code>RULES[]</code> in <code>grammars.ts</code> and <code>FRAMES</code> per difficulty in <code>topics.ts</code> select behaviour by data, not branching logic.              |
| Adapter                         | Yes      | <code>useSpeechRecognition</code> adapts the messy Web Speech API into a clean React interface; <code>applyEnhancement</code> adapts LLM JSON into <code>CoachReport</code> . |
| Custom Hook (React idiom)       | Yes      | <code>useSpeechRecognition</code> encapsulates stateful behaviour for reuse.                                                                                                  |
| State Machine                   | Yes      | <code>Screen</code> union in <code>App.tsx</code> (finite, explicit transitions).                                                                                             |
| Factory (light)                 | Partial  | <code>generateTopic()</code> / <code>generateChallengeTopic()</code> construct <code>Topic</code> objects.                                                                    |
| Null Object / graceful fallback | Partial  | <code>enhanceWithLlm</code> returns <code>null</code> □ caller keeps deterministic report.                                                                                    |
| MVC                             | Loose    | Model= <code>lib</code> + <code>types</code> , View= <code>components</code> , Controller= <code>App</code> handlers — not a formal MVC framework.                            |
| Singleton                       | No       | Module-scope constants ( <code>SEEDS</code> , <code>FILLER_SET</code> ) are effectively single instances, but no Singleton class.                                             |
| Observer                        | No       | React's own reactivity aside, no pub/sub in app code.                                                                                                                         |
| Dependency Injection            | No       | Dependencies are imported directly (see DIP note in §14).                                                                                                                     |
| Decorator / Command / Builder   | No       | Not used.                                                                                                                                                                     |

## 16

## Libraries

The dependency footprint is intentionally tiny. Runtime deps: **3**.

| Dependency                                                   | Type    | Why it exists / why chosen                                | Alternatives                                   | Trade-offs                                                            |
|--------------------------------------------------------------|---------|-----------------------------------------------------------|------------------------------------------------|-----------------------------------------------------------------------|
| <code>react</code> ^18.3                                     | runtime | Declarative UI; concurrent-ready; team-standard           | Vue, Svelte, SolidJS, Preact                   | +Ecosystem/hirability; -larger than Svelte/Preact                     |
| <code>react-dom</code> ^18.3                                 | runtime | DOM renderer + <code>createRoot</code>                    | (paired with React)                            | Required with React on web                                            |
| <code>recharts</code> ^2.12                                  | runtime | Declarative React charts (radar + line) with minimal code | Chart.js, visx, nivo, ECharts, hand-rolled SVG | +Fast to build; <b>-bundle size</b> (D3 deps) — main perf cost (\$13) |
| <code>vite</code> ^5.4                                       | dev     | Fast dev server + optimized Rollup build                  | CRA (deprecated), Webpack, Parcel, esbuild     | +DX/speed; -younger plugin ecosystem than Webpack                     |
| <code>@vitejs/plugin-react</code>                            | dev     | React Fast Refresh + JSX transform for Vite               | SWC plugin variant                             | Standard choice                                                       |
| <code>typescript</code> ^5.5                                 | dev     | Static types; the domain model is the backbone of the app | JSDoc + JS, Flow                               | +Safety/refactoring; -build step                                      |
| <code>@types/react</code> ,<br><code>@types/react-dom</code> | dev     | Type definitions for React                                | —                                              | Required for TS                                                       |

**Notable absences** (by design): no state library (Redux/Zustand), no router, no form/validation lib (Formik/RHF/Zod), no HTTP client (uses `fetch`), no UI kit (hand-rolled CSS), no test framework. Fewer deps = smaller attack surface and simpler mental model — appropriate for this scope, though tests should be added.

## 17

## Deployment

## 17.1 Build &amp; scripts

| Command                      | Effect                                                                             |
|------------------------------|------------------------------------------------------------------------------------|
| <code>npm run dev</code>     | Vite dev server on <code>:5173</code> (HMR)                                        |
| <code>npm run build</code>   | <code>tsc -b</code> (type-check) then <code>vite build</code> → <code>dist/</code> |
| <code>npm run preview</code> | Serve the built <code>dist/</code> locally                                         |
| <code>npm run lint</code>    | <code>tsc --noEmit</code> (type-only lint; no ESLint configured)                   |

## 17.2 Environments &amp; env vars

**No runtime environment variables.** The OpenAI key is entered by the user at runtime (not an `.env`). Configuration is entirely build-time ( `vite.config.ts` , `tsconfig*.json` ). There is no separate dev/staging/prod config in the repo.

## 17.3 Hosting

`dist/` is fully static, so any static host works: **Netlify, Vercel, GitHub Pages, Cloudflare Pages, or S3+CloudFront**. SPA fallback (serve `index.html` for unknown paths) is trivial but, given the state-machine routing, not strictly required.

17.4 CI/CD, Docker, monitoring, logging NOT PRESENT

- **CI/CD:** no `.github/workflows` , no pipeline. Deploy is manual ( `build` → upload ).
- **Docker:** no `Dockerfile` (a static SPA rarely needs one; an nginx image is optional).
- **Monitoring/logging:** none. **Fix:** add web-vitals + an error reporter (e.g. Sentry) and basic product analytics.

**Recommended minimal pipeline:** GitHub Actions → `npm ci` → `npm run build` (type-check gate) → deploy `dist/` to the static host on merge to `main` ; add a preview deploy per PR.

## 18

# Complete Interview Questions

Questions are grounded in this codebase and grouped by category. Each carries a difficulty pill and, where useful, *why it's asked*, an *ideal answer*, *common mistakes*, and *follow-ups*. Difficulty legend: Easy Medium Hard Expert Staff

**Principal**

## 18.1 Project Introduction

### Q1. Walk me through what this project does. Easy

*Why:* opener; checks you can summarise scope crisply.

**Ideal:** "It's a client-side React/TS SPA that coaches spoken English. You pick a GD/interview topic, speak against a timer while the Web Speech API transcribes you, and a deterministic engine returns a scored coaching report — 13 dimensions, grammar fixes, WPM, fillers, structure — plus gamification and a progress dashboard. An optional LLM layer adds qualitative prose."

**Follow-ups:** Why client-side? What's deterministic vs. AI? What did you personally build?

### Q2. What was the hardest technical problem you solved here? Medium

**Ideal:** "Two: (1) making the analysis *honest and testable* — a pure `analyzeSpeech()` that never fabricates scores, guarded by `MIN_SPEECH_WORDS`; (2) taming the Web Speech API, which auto-stops in Chrome mid-speech — I keep a `shouldListenRef` and restart on `onend`."

**Follow-ups:** How did you verify the engine? What happens on empty speech?

### Q3. If you had one more week, what would you build? Easy

**Ideal:** "A backend proxy to hide the OpenAI key (my top security gap), then Vitest tests around the engine, then route-based code-splitting to cut the ~610 kB bundle."

## 18.2 Architecture

### Q4. Describe the architecture in one minute. Medium

**Ideal:** "A frontend monolith with three layers: presentation (components/), orchestration (`App.tsx` + hooks), and a pure domain layer (`lib/`). Imports flow one way, components `□` lib `□` types. Persistence is a small repository over `localStorage`. There is no backend — scoring runs in the browser."

**Follow-ups:** Where's the boundary you'd add a server? How do modules depend on each other?

### Q5. Why no backend? Isn't that a limitation? Hard

*Why:* tests whether you can defend a decision and know its costs.

**Ideal:** "It's a deliberate MVP trade-off. Deterministic scoring doesn't *need* a server, so going client-only gave zero infra cost, instant privacy, and offline capability. The costs are real and I can name them: no cross-device sync, no way to hide the API key, and no server-side rate limiting. The first thing I'd add for production is a thin proxy — that single component resolves the key-exposure and rate-limit gaps at once."

**Common mistake:** claiming there are no downsides. Always volunteer the trade-off.

**Q6. How is the analysis engine decoupled from React?** Medium

**Ideal:** "lib/analysis/\* is plain TypeScript with no React import. analyzeSpeech(input): CoachReport is a pure function. That's why scripts/smoke.ts can run it in Node with no DOM — and why it's trivially unit-testable."

**Q7. Design a v2 with accounts and cloud sync.** Staff

**Ideal:** "Add one stateless service (Node/serverless) with two concerns: /api/enhance proxying OpenAI (secret key, Redis rate-limit + cache) and /api/sessions CRUD backed by Postgres (users, sessions with a JSONB report). Auth via Google OAuth + short-lived JWT and an httpOnly refresh cookie; row-level security so users touch only their rows. Keep the client engine as-is for offline-first, sync sessions opportunistically (AP, eventual consistency). CDN stays in front of static assets."

**Follow-ups:** Why Postgres over Mongo? Why AP not CP? How do you migrate existing localStorage users?

## 18.3 Frontend / React

**Q8. How does navigation work without a router?** Easy

**Ideal:** "A discriminated union type Screen in App.tsx plus useState; each screen is conditionally rendered. It's a finite state machine with explicit transitions via setScreen()."

**Follow-up:** What do you lose? (deep-linking, back button, refresh state) When would you switch to React Router?

**Q9. Why no Redux/Zustand/Context?** Medium

**Ideal:** "State is shallow and owned by one component; lifting state up + props/callbacks is sufficient and has zero overhead. Introducing a store would be premature abstraction. If the tree deepened or many distant components needed the same state, I'd reach for Context or Zustand."

**Common mistake:** saying Redux is always better. Right answer is 'fit to complexity'.

**Q10. Explain your custom hook and why useRef is used there.** Hard

**Ideal:** "useSpeechRecognition wraps the Web Speech API. I use useRef for the recognizer instance and a shouldListenRef boolean because these must persist across renders *without* triggering re-renders — a ref is mutable, stable storage. State (listening, finalTranscript) is separate because it *should* re-render the UI. On onend, if we still want to listen, I restart recognition — that's how I handle Chrome's periodic auto-stop."

**Follow-ups:** Why not put the recognizer in state? What cleanup do you do on unmount? What if two start() calls race?

**Q11. How do you prevent stale closures in the countdown timer?** Hard

**Ideal:** "The interval updates remaining with the functional updater setRemaining(r => ...), so it never closes over a stale value. The timer id and start time are refs so the effect cleanup can clear the interval reliably."

**Q12. What re-renders when the timer ticks, and is that a problem?** Medium

**Ideal:** "Only SpeakingScreen re-renders (other screens are unmounted). It's a tiny subtree, so a 1 Hz re-render plus transcript updates is negligible. If it mattered, I'd isolate the timer display into a memoised child."

**Q13. Why React.StrictMode, and what does it change? Medium**

**Ideal:** "It double-invokes renders/effects in dev to surface impure logic and missing cleanups. It's why robust effect cleanup (aborting recognition, clearing intervals) matters — StrictMode mounts, unmounts, and remounts."

**Q14. How would you code-split this app? Medium**

**Ideal:** "Wrap ReportView and Dashboard in `React.lazy(() => import(...))` with `Suspense`, so Recharts (the bulk of the ~610 kB) only downloads when a chart is actually shown. Add `manualChunks` to separate vendor from app code."

**Q15. There's no Error Boundary — what's the risk and fix? Medium**

**Ideal:** "A render-time throw anywhere would unmount the whole tree (blank screen). Fix: a class `Error Boundary` around the screen switch that shows a fallback and a 'reset' action; optionally report to Sentry."

## 18.4 TypeScript

**Q16. How do DURATION\_OPTIONS and its Duration type work? Medium**

**Ideal:** "DURATION\_OPTIONS = [30,45,...] as `const` is a readonly tuple of literals; `type Duration = typeof DURATION_OPTIONS[number]` derives the union `30 | 45 | 60 | ...`. One array is both the runtime source of the UI buttons and the compile-time type — no drift between them."

**Follow-up:** Why as `const`? (widening to `number[]` otherwise)

**Q17. Where do you use discriminated unions? Medium**

**Ideal:** "Screen, Difficulty, SessionMode are string-literal unions that make invalid states unrepresentable and give exhaustive switch checking (helped by `noFallthroughCasesInSwitch`)."

**Q18. What strictness flags are on and why do they matter? Hard**

**Ideal:** "strict, noUnusedLocals, noUnusedParameters, noFallthroughCasesInSwitch. They catch null/undefined bugs, dead variables, and missing switch cases at compile time. The build gate is `tsc -b`, so type errors fail the build."

**Q19. How is the optional LLM field typed without polluting the base type? Hard**

**Ideal:** "CoachReport.llmDetailedFeedback?: string is optional, and `llmEnhanced`: boolean flags provenance. The deterministic path sets `llmEnhanced:false` and omits the optional field; `applyEnhancement` returns a new object with them set — no mutation, and the UI can branch on `llmEnhanced`."

## 18.5 The Analysis Engine (domain deep-dive)

**Q20. How is WPM computed, and what's the edge case? Medium**

**Ideal:** "wpm = round(totalWords / minutes) where minutes = max(durationSec/60, 1/60). The max avoids divide-by-zero for a sub-second duration. It uses `actualSpeakingSec`, not the target, so a user who finishes early isn't penalised on pace."

**Follow-up:** Why actual vs target seconds?



**Q21. How are filler words detected, including multi-word ones?** Medium

**Ideal:** "Single-word fillers: tokenise, then membership-test against a Set ( $O(1)$  lookups). Multi-word phrases ('you know', 'i mean'): build a whitespace-tolerant regex per phrase and count matches on the raw lowercased text. Results are merged into one count map and sorted."

**Follow-up:** Why a Set for single words but regex for phrases?

**Q22. How is vocabulary diversity measured?** Medium

**Ideal:** "Type-token ratio:  $\text{uniqueWords} / \text{totalWords}$  using a Set for the numerator. Higher ratio = less repetition. It also finds overused words ( $\text{count} \geq 3$ ), flags weak words with upgrade suggestions, and detects 'advanced' words used."

**Follow-up (trap):** Isn't TTR biased by length? (Yes — short samples score higher; a real system would use MTLD/moving-average TTR.)

**Q23. Explain the grammar checker design. Why regex and not an NLP parser?** Hard

**Ideal:** "It's a Strategy list: an array of {test, fix, explanation, rule, difficulty} rules applied per sentence. Regex was chosen for zero dependencies, determinism, speed, and offline use. It catches high-frequency ESL errors ('should of', 'he don't', 'didn't went', a/an). The honest limitation is recall — it's not a parser, so it misses novel errors. That's exactly why the optional LLM exists for deeper feedback."

**Follow-ups:** How do you avoid duplicate issues? (a seen Set keyed by rule+sentence) How do you extend it? (append a rule object — open/closed)

**Q24. How does structure detection work and what's its weakness?** Medium

**Ideal:** "Discourse-marker lexicons (intro/argument/example/counter/conclusion) are substring-matched against the lowercased transcript; presence of each contributes to a 0–100 structure score. Weakness: marker-presence is a proxy — a well-structured speech that avoids explicit markers could score lower. It's a pragmatic heuristic, and I'd label it as such."

**Q25. Walk me through the scoring for confidence. Why did empty speech used to score high?** Hard

**Ideal:** " $\text{confidence} = \text{clamp}(92 - \text{fearSignals} * 12 - \text{fillerRate} * 150 - \text{repetitionRatio} * 20)$ . With an empty transcript, fillerRate and repetition are 0 and few fear signals fire, so it started near 92 — a bug that gave 'marks for silence'. I fixed it with a `MIN_SPEECH_WORDS = 12` gate in `App.handleFinish`: below the threshold we route to a 'no speech' screen and never score, save, or award XP."

**Follow-ups:** Why guard in the app rather than zero the scores in the engine? Would you make the engine reject too? (I added `hasSufficientSpeech` there as well.)

**Q26. How does difficulty affect scores?** Easy

**Ideal:** "A `difficultyPenalty` (Advanced -4, Expert -8) is subtracted from every dimension via an `adj()` wrapper, so harder settings grade more strictly."

**Q27. The engine has `void expectedWordsPerSentence` — what is that?** Medium

**Ideal:** "Honest answer: leftover from an earlier pause-estimation approach. The variables are computed but discarded via `void` to satisfy `noUnusedLocals`. It's dead code I'd remove in cleanup — I flagged it in my own code review (§14)."

**Why asked:** integrity — do you own imperfections in your code?

## 18.6 Data Structures, Algorithms & Complexity

### Q28. What data structures does the engine use and why? Medium

**Ideal:** "Set for  $O(1)$  filler membership and for de-duplication (unique words, seen grammar issues); Map for word-frequency counting (countBy) and filler tallies; arrays for ordered lists and sorting. The choices are driven by the operation: membership  $\square$  Set, counting  $\square$  Map, ordering  $\square$  array."

### Q29. What's the time complexity of analyzeSpeech? Medium

**Ideal:** " $O(n)$  in transcript length. Tokenising is one pass; frequency counting is  $O(w)$ ; grammar is 10 (constant) rules  $\times$  sentences; structure markers are a constant set of substring scans; scoring is  $O(1)$  over 13 fixed dimensions. Space is  $O(w)$  for the maps/sets."

**Follow-up:** Any hidden quadratics? (The per-phrase regex and per-marker includes are constant-count  $\times n$ , so still linear.)

### Q30. How does the level-up loop work and could it be $O(1)$ ? Hard

**Ideal:** "while (xp >= level\*100) { xp -= level\*100; level++ }. It's a loop but bounded by how many levels a single session's XP can cross (tiny), so effectively  $O(1)$ . With a closed-form threshold you could compute the target level directly, but the loop is clearer and cheap."

### Q31. Explain the syllable estimator. Is it exact? Hard

**Ideal:** "It strips silent endings ('e', 'es', 'ed'), removes a leading 'y', then counts vowel groups via regex, min 1. It's a well-known heuristic — fast and dependency-free but not exact (English is irregular). It's only used to flag 'likely hard to pronounce' long words, so approximate is fine."

## 18.7 Data / localStorage / "Database"

### Q32. How is data persisted and modelled? Easy

**Ideal:** "Three localStorage keys (sessions, gamification, api key), read/written through a repository in storage.ts. Sessions embed their topic and full report (document-style denormalisation), so one read renders everything."

### Q33. What are the risks of localStorage here? Medium

**Ideal:** "~5 MB cap (unbounded history could hit it), synchronous main-thread API, no cross-tab transactions, no schema enforcement on read, and it's plaintext (bad for the key). I mitigate parse failures with safeParse; I'd add pruning, a schema version + migration, and Zod validation."

### Q34. Your session write and gamification write aren't atomic. Why does that matter? Hard

**Ideal:** "They're two separate setItem calls. Each is atomic alone, but a crash between them could leave XP updated without the session saved (or vice-versa) — a consistency gap. On a real DB I'd wrap both in a transaction; on the client I could write a single combined key or reconcile on load."

**Why asked:** do you understand atomicity/transactions beyond the buzzword?

**Q35. If you moved to Postgres, what's the schema?** Staff

**Ideal:** "users(id, ...); sessions(id, user\_id FK, created\_at, mode, difficulty, duration\_sec, actual\_sec, transcript, report JSONB, overall\_score). Index (user\_id, created\_at) for the dashboard timeline and a generated column or index on overall\_score for best/avg. Report stays JSONB because it's read as a whole; I'd promote overall\_score and a few scores to columns for aggregation."

## 18.8 Async, Promises & Concurrency

**Q36. Why is `handleFinish` async?** Easy

**Ideal:** "Because the optional LLM call is awaited. The deterministic report is computed synchronously first; only the enhancement is async. If no key, nothing is awaited and it proceeds immediately."

**Q37. What concurrency bug did the Web Speech API create and how did you fix it?** Hard

**Ideal:** "Chrome fires `onend` unpredictably mid-speech. Naively that stops capture. I restart on `onend` while `shouldListenRef.current` is true, and wrap `start()` in `try/catch` because calling `start()` on an already-started recognizer throws. On `unmount` I flip the flag and `abort()` to avoid a zombie recognizer."

**Follow-up:** race between user 'stop' and an in-flight restart? (the flag is checked in `onend`, so stop wins.)

**Q38. How do you keep the UI responsive during analysis?** Medium

**Ideal:** "Deterministic analysis is  $O(n)$  and sub-millisecond, so it runs inline. An analyzing flag shows a spinner while the awaited LLM call resolves. For very heavy future analysis I'd move it to a Web Worker to keep the main thread free."

## 18.9 AI / LLM / Prompt Engineering

**Q39. How is the LLM integrated and why is it optional?** Medium

**Ideal:** "It's purely additive: `enhanceWithLlm` is called only if a key exists, and it can only *augment* the report's prose via `applyEnhancement`. Any failure returns null and the deterministic report stands. The product never depends on the model being up or correct."

**Q40. How do you get reliable structured output from the model?** Hard

**Ideal:** "Two layers: the API's JSON mode (`response_format: {type:'json_object'}`) and a prompt that specifies the exact JSON shape. Then I still validate defensively — `??` " defaults and `Array.isArray` guards — because 'valid JSON' isn't 'the schema I expected'."

**Follow-up:** Why not trust JSON mode alone? (It guarantees JSON, not your schema.)

**Q41. How do you prevent the LLM from duplicating the deterministic metrics?** Hard

**Ideal:** "The system prompt tells it the metrics are *already computed* and to add human, specific judgement — 'NOT a grammar checker'. I also feed it the engine's score/WPM/fillers as context so it aligns rather than recomputes."

**Q42. Prompt injection — is the transcript a risk?** Expert

**Ideal:** "The transcript is untrusted and sits in the user message, so it could try 'ignore your instructions'. Blast radius is low: output is schema-constrained, shown only to the same user, and never executed or used to take actions. I'd still (a) keep JSON mode, (b) validate output shape, (c) treat transcript strictly as data, and (d) never feed model output into privileged operations."

**Q43. Estimate tokens and cost per enhanced session.** Hard

**Ideal:** "~250 system + a few hundred transcript tokens in (<~800), ~150–300 out. On gpt-4o-mini that's a fraction of a cent. To control cost at scale: proxy + cache identical transcripts, make it opt-in ('deeper feedback'), cap transcript length, and tier models."

## 18.10 Security

**Q44. What's the biggest security issue and how do you fix it?** Hard

**Ideal:** "The OpenAI key is used directly from the browser, so it's exposed in memory and network requests. Fix: a backend proxy that holds the key server-side; the browser only sends the transcript. That also gives per-user rate limiting and caching. It's the #1 production change."

**Common mistake:** saying 'store it in .env' — a client-side .env is still bundled and public.

**Q45. Is this app vulnerable to XSS?** Medium

**Ideal:** "Low risk: React escapes interpolated text by default, there's no dangerouslySetInnerHTML, no eval, and user text (transcript) is rendered as text. I'd still add a strict CSP at the host as defence-in-depth."

**Q46. Why are CSRF and SQL injection not concerns here?** Medium

**Ideal:** "No cookies/sessions and no state-changing first-party endpoints → no CSRF surface. No database and no SQL → no SQL injection. They become relevant only when I add the backend, at which point parameterised queries and anti-CSRF tokens/SameSite cookies apply."

## 18.11 System Design, Caching, Redis, Queues, Microservices, Docker, Cloud

**Q47. How does this scale to a million users?** Hard

**Ideal:** "The core is static assets + client compute, so a CDN handles a million users trivially. Only the optional AI proxy needs classic scaling — and it's stateless, so it scales horizontally behind a load balancer with Redis for rate-limiting and response caching. Data would move to Postgres with read replicas; LLM calls get a queue with backpressure for cost control."

**Q48. Where would Redis help?** Medium

**Ideal:** "At the future proxy: (1) token-bucket rate limiting per user, (2) caching LLM responses keyed by a transcript hash to cut cost/latency, (3) optionally short-lived session/token storage."

**Q49. Do you need microservices or a queue?** Hard

**Ideal:** "No — that would be over-engineering. One stateless service covers enhance + sessions. A queue only earns its place if AI calls become batched/async or need retry with backpressure; then a job queue in front of OpenAI makes sense."

**Q50. Would you Dockerise it?** Easy

**Ideal:** "The SPA is static — it doesn't need a container to run; a CDN is simpler. I'd Dockerise the future backend proxy for consistent deploys. Optionally an nginx image serves the static bundle if the platform requires containers."

## 18.12 Testing, Git, CI/CD

**Q51. How is this tested? Be honest.** Medium

**Ideal:** "Honestly, there's no automated test suite yet — just scripts/smoke.ts, a manual functional check that runs analyzeSpeech on a known transcript and asserts WPM/fillers/grammar/structure. The engine is pure, so it's *designed* for unit testing; the next step is Vitest for lib/ and React Testing Library for screens, gated in CI."

**Why asked:** integrity + knowing the gap. Never claim tests that don't exist.

**Q52. What would your first 5 unit tests be?** Medium

**Ideal:** "(1) WPM for known words/seconds; (2) filler counting incl. a multi-word phrase; (3) each grammar rule fires + corrects; (4) hasSufficientSpeech boundary at 11/12 words; (5) awardForSession streak logic for gap 0/1/>1 days (with a mocked store)."

**Q53. Describe your Git workflow on this repo.** Easy

**Ideal:** "Feature branch off main, focused commits with descriptive messages, PR for review, never push straight to main. I keep build artifacts out via .gitignore (dist, \*.tsbuildinfo)."

**Q54. What CI/CD would you add?** Medium

**Ideal:** "GitHub Actions: npm ci □ typecheck (tsc -b) □ tests □ vite build □ deploy dist/ to the static host on merge; PR preview deploys. Add ESLint + Prettier as gates."

## 18.13 JavaScript / Node fundamentals (surfaced by this code)

**Q55. Why functional setState updaters in the timer?** Medium

**Ideal:** "To avoid stale closures: setRemaining(r => r-1) uses the latest value rather than the value captured when the interval was created."

**Q56. Explain a ?? b vs a || b as used in the code.** Easy

**Ideal:** "?? only falls back on null/undefined; || falls back on any falsy (0, '', false). loadApiKey uses ?? " so a legitimately empty stored value is handled correctly; scoring code intentionally uses || -style OR only where falsy really means 'absent'."

**Q57. What does `as const` do and where is it critical?** Medium

**Ideal:** "Freezes literals and narrows types; critical for `DURATION_OPTIONS` so its element type is a literal union, not number (Q16)."

## 18.14 Behavioral / Ownership / Leadership

**Q58. Tell me about a bug you found and fixed here.** Easy

**Ideal (STAR):** "*Situation:* testing revealed the app scored a user who said nothing. *Task:* stop 'marks for silence' without hurting real short answers. *Action:* traced it to base scores (confidence started ~92); added `MIN_SPEECH_WORDS=12` and a guard in `handleFinish` plus `hasSufficientSpeech` in the engine, routing to a dedicated screen and skipping scoring/saving/XP. *Result:* verified with boundary cases (0/3/15 words); trust restored."

**Q59. Tell me about a trade-off you made and would defend.** Medium

**Ideal:** "Client-only, deterministic-first. I chose zero infra + privacy + offline over cross-device sync and hidden keys, then documented the exact production path to close those gaps. I'd defend it because it shipped a working, honest product immediately while keeping a clean upgrade route."

**Q60. How did you ensure quality without a QA team?** Medium

**Ideal:** "Strict TypeScript as a first gate, a pure engine I could smoke-test in Node, and a build that fails on type errors. I also self-reviewed (see the code-review section) and listed the debt honestly rather than hiding it."

**Q61. Disagree-and-commit: someone insists on Redux here.** Hard

**Ideal:** "I'd show that state is shallow and local, so Redux adds boilerplate without benefit, and propose a lightweight Context/Zustand only if we hit prop-drilling pain. If the team still wanted it for consistency across a larger codebase, I'd commit and implement it cleanly — the decision is reversible and low-risk."

## 18.15 Trap, Cross & "What-if" Questions

**Q62. You said it's "AI-powered". But the scores aren't AI at all, are they?** Expert · TRAP

**Ideal:** "Correct, and I'd own that framing. The *scoring* is deterministic heuristics, not AI — that's a feature (cheap, offline, testable, honest). The AI is an *optional enhancement* for qualitative prose. If a JD demands 'AI', I'd describe it precisely rather than overclaim; overclaiming is exactly what a good interviewer is probing for."

**Trap avoided:** don't pretend the heuristics are ML.

**Q63. Your grammar checker will miss most real errors. Isn't it useless?** Hard · CROSS

**Ideal:** "It has high precision on common ESL patterns and zero cost/latency, which is valuable for instant feedback. It's not meant to be comprehensive — recall is intentionally traded for determinism, and the LLM path covers depth. Calling it 'useless' ignores the precision/latency/cost trade-off it's optimised for."

**Q64. What if two browser tabs run sessions simultaneously?** Hard · WHAT-IF

**Ideal:** "Both read the same localStorage on load and last-writer-wins on save, so one tab's session list could clobber the other's (no locking, no storage-event sync). Today it's an edge case; fixes are listening to the storage event to reconcile, or moving to a backend with proper concurrency control."

**Q65. What if the user speaks a non-English language?** Medium · WHAT-IF

**Ideal:** "Recognition is hard-set to en-US, and the lexicons/grammar rules are English-only, so results would be poor/meaningless. I'd need per-language recognizer config and localized rule/lexicon sets — a real internationalisation effort, currently out of scope."

**Q66. What if localStorage is full or disabled?** Medium · WHAT-IF

**Ideal:** "setItem throws QuotaExceededError; today that isn't caught around saveSession, so it could bubble. I'd wrap writes in try/catch, surface a 'storage full' notice, and offer pruning/export. If storage is disabled entirely, reads fall back to defaults via safeParse but nothing persists."

**Why asked:** do you know the failure modes of your own persistence layer?

**Q67. Why should I trust a score if you can't even measure pronunciation from audio?** Expert · TRAP

**Ideal:** "You shouldn't treat the pronunciation number as ground truth — and the UI says exactly that, labelling it 'estimated from text, not audio'. I chose transparency over false precision. The metrics that *are* real (WPM, fillers, structure) are clearly derived from the transcript+timing. Honesty about measurement limits is the trust mechanism."

**Q68. If I remove text.ts, what breaks and why?** Medium · WHAT-IF

**Ideal:** "The engine and grammar both import primitives (words, sentences, clamp, countBy, syllables) from it, so analysis collapses. It's the shared foundation of the domain layer — high fan-in, which is why it's tiny and pure."

**Q69. Why is the bundle 600 kB for such a small app?** Medium · CROSS

**Ideal:** "Recharts (and its D3 dependencies) dominate, and it's currently in the main chunk even though charts only appear on the report/dashboard screens. Fix: lazy-load those screens; the home/speaking flow then ships almost none of it."

**Q70. Principal-level: critique your own architecture.** Principal

**Ideal:** "Strengths: clean layering, a pure testable core, honest UX, graceful degradation. Weaknesses I'd prioritise: (1) the client-side key is a security boundary I traded away — fix with a proxy; (2) App.tsx is a god component mixing state, orchestration and view — extract a session hook + router; (3) hidden side-effect coupling (gamification ↔ storage) hurts testability — invert the dependency; (4) no tests/CI and magic-number scoring constants — extract a tuning module and pin behaviour with tests. None are architectural dead-ends; they're a prioritised backlog with clear ROI."

## 19

## Live Coding Questions

Exercises derived directly from this codebase. Each states the task, the expected approach, and a reference solution consistent with the existing style.

### LC1. Word-frequency & top-N repeated words Easy

**Task:** Given a transcript, return the top 3 words (length > 3) by frequency. (This is exactly what `analyzeVocabulary` does.)

```
function topRepeated(text: string, n = 3) {
  const counts = new Map<string, number>();
  for (const w of text.toLowerCase().match(/[a-z']+/g) ?? []) {
    if (w.length > 3) counts.set(w, (counts.get(w) ?? 0) + 1);
  }
  return [...counts.entries()]
    .sort((a, b) => b[1] - a[1])
    .slice(0, n)
    .map(([word, count]) => ({ word, count }));
}
```

**Discussion:** Map for O(1) counting; sort is O(k log k) over unique words. Interviewers probe: tokenisation choice, tie-breaking, and why not sort the raw array.

### LC2. Words-per-minute with a safe divisor Easy

**Task:** Compute WPM from word count and seconds, avoiding divide-by-zero.

```
const wpm = Math.round(totalWords / Math.max(seconds / 60, 1 / 60));
```

**Follow-up:** Why clamp the divisor rather than special-case `seconds === 0` ? (One expression, no branch, always defined.)

### LC3. A pluggable rule engine (Strategy) Medium

**Task:** Design a grammar checker that's easy to extend without editing logic — then add a rule. (Mirrors `grammars.ts`.)



```
interface Rule { test: RegExp; fix: (s: string) => string; rule: string; }
const RULES: Rule[] = [
  { test: /\b(should|could|would)\s+of\b/i,
    fix: s => s.replace(/\b(should|could|would)\s+of\b/gi, "$1 have"),
    rule: "Modal + have" },
];
function check(text: string) {
  const out: {original:string; corrected:string; rule:string}[] = [];
  for (const s of text.split(/(?<=[.!?])\s+/)) {
    for (const r of RULES) {
      if (r.test.test(s)) {
        const corrected = r.fix(s);
        if (corrected.toLowerCase() !== s.toLowerCase())
          out.push({ original: s, corrected, rule: r.rule });
      }
    }
  }
  return out;
}
```

**Expected talking points:** open/closed principle, de-duplication with a `Set`, why regex over parsing, precision vs recall.

#### LC4. Streak counter Medium

**Task:** Update a daily streak given the last-session date and today (mirrors `awardForSession`).

```
function nextStreak(streak: number, last: Date | null, today: Date): number {
  if (!last) return 1;
  const ms = 864e5;
  const d = Math.round(
    (Date.UTC(today.getFullYear(), today.getMonth(), today.getDate()) -
     Date.UTC(last.getFullYear(), last.getMonth(), last.getDate())) / ms);
  if (d === 0) return streak; // same day
  if (d === 1) return streak + 1; // consecutive
  return 1; // gap → reset
}
```

**Trap:** using raw `Date` subtraction (DST/timezones) instead of UTC-midnight normalisation.

#### LC5. React.lazy the chart screens Medium

**Task:** Reduce first-load bundle by lazy-loading `ReportView` / `Dashboard`.

```
const ReportView = React.lazy(() => import("../components/ReportView"));
// ...
{screen === "report" && report && (
  <Suspense fallback={<div className="card center">Loading...</div>}>
    <ReportView ... />
  </Suspense>
)}
```

**Note:** requires converting those components to default exports (they're currently named exports).

## LC6. Debounced autosave (design) Hard

**Task:** If sessions were saved on every keystroke of a manual transcript, debounce writes. Expected: a `useEffect` + `setTimeout` cleared on change, or a `useDebounceCallback`; discuss trailing vs leading edge and cancellation on unmount.

## 20

# Resume Verification

Claims you might put on a resume, checked against the code, with honest ratings and stronger wording.

| Draft bullet                                                  | Verdict      | Why                                                                                     |
|---------------------------------------------------------------|--------------|-----------------------------------------------------------------------------------------|
| "Built an AI-powered speaking coach"                          | Partly true  | Core scoring is deterministic; AI is optional. Risk of overclaim (see Q62).             |
| "Developed a deterministic NLP analysis engine in TypeScript" | Accurate     | <code>engine.ts</code> + <code>grammar.ts</code> + <code>lexicons.ts</code> back this.  |
| "Designed a scalable microservices backend"                   | False        | No backend exists. Do NOT write this.                                                   |
| "Integrated OpenAI with graceful fallback"                    | Accurate     | <code>enhanceWithLlm</code> returns <code>null</code> on failure; report still renders. |
| "Implemented real-time speech-to-text"                        | Needs nuance | Used the Web Speech API (didn't implement STT); say "integrated".                       |
| "Achieved 90%+ test coverage"                                 | False        | Only a manual smoke script exists.                                                      |

## 20.1 Stronger, defensible bullets

- "Architected a client-only React 18 + TypeScript SPA whose **pure, dependency-free analysis engine** scores speech across 13 dimensions in  $O(n)$  time — fully offline and unit-testable."
- "Designed a **data-driven rule engine** (Strategy pattern) for grammar detection and a discourse-marker structure analyser, keeping runtime dependencies to three."
- "Integrated the browser **Web Speech API** behind a custom hook that handles Chrome's mid-speech auto-stop via a restart-on-`onend` pattern."
- "Added an **optional OpenAI enhancement** with strict JSON-mode parsing and graceful degradation, so the product never depends on model availability."
- "Identified and fixed a scoring integrity bug ('marks for silence') with a `MIN_SPEECH_WORDS` guard, verified across boundary cases."

## 20.2 Weak points to pre-empt (and your prepared answers)

- **No tests/CI** □ "Engine is pure and smoke-tested; next step is Vitest + Actions."
- **Client-side key** □ "Known high-sev gap; fix is a proxy (I can whiteboard it)."
- **Heuristics, not ML** □ "Deliberate: cheap, offline, honest; LLM covers depth."

## 21

## Mock Interview (Full Transcript)

A single escalating interview — warm-up → deep dive → system design → cross-examination → behavioral — with the answers a strong candidate would give. **I** = Interviewer, **C** = Candidate.

**I:** Give me the 30-second version of your project.

**C:** "SpeakX AI is a client-only React + TypeScript SPA that coaches spoken English. You pick a GD or interview topic, speak against a timer while the browser transcribes you, and a deterministic engine returns a scored coaching report — 13 dimensions, grammar fixes, words-per-minute, fillers, structure — plus gamification and a progress dashboard. There's an optional LLM layer for richer prose, but the app is fully functional without it."

**I:** "Deterministic engine" — unpack that. What's actually computed?

**C:** WPM from words over actual speaking seconds; single-word fillers via a `Set` and multi-word fillers via regex; vocabulary diversity as a type-token ratio; ten regex grammar rules; and structure via discourse-marker lexicons. Everything is a pure function in `lib/analysis/engine.ts` returning a typed `CoachReport`.

**I:** Pure function — prove it's testable.

**C:** It imports no React and no DOM, so `scripts/smoke.ts` runs it in Node with a fixed transcript and asserts WPM, filler counts, that grammar rules fire, and structure flags. That's also why unit tests are the obvious next step.

**I:** I typed nothing and still got a score once. Bug?

**C:** Yes — a real one I fixed. Base scores started high (confidence ~92), so silence scored well. I added `MIN_SPEECH_WORDS = 12`; `handleFinish` checks `countSpokenWords` and, below the threshold, routes to a 'no speech' screen without scoring, saving, or awarding XP. I also added `hasSufficientSpeech` in the engine and verified 0/3/15-word cases.

**I:** Why did you add the guard in the component, not the engine?

**C:** The engine stays a pure calculator — given input, it computes. The *policy* decision "don't record a non-attempt" is an app concern, so it lives in `handleFinish`. I mirrored a check in the engine too, so a future caller can't misuse it.

**I:** Let's design v2 with accounts and cloud sync. Whiteboard it.

**C:** One stateless service. `/api/enhance` proxies OpenAI so the key is a server secret — that closes my biggest security gap — with Redis for per-user rate limiting and caching keyed by transcript hash. `/api/sessions` is CRUD over Postgres: `users` and `sessions(user_id, created_at, report JSONB, overall_score)`, indexed on `(user_id, created_at)`. Auth via Google OAuth + short-lived JWT and an httpOnly refresh cookie; row-level security. Keep the client engine for offline-first and sync sessions opportunistically — AP with eventual consistency, since there are no concurrent writers to a record.

**I:** Why AP and not strong consistency?

**C:** A user must always be able to record a session, even offline (availability). Their history is single-writer, so a few seconds of cross-device staleness is harmless. Forcing CP would hurt UX for no correctness benefit.

**I:** Postgres over Mongo — defend it.

**C:** The report is already a document, so Mongo would work, but I want relational reporting — averages, best/worst, trends per user — which is cleaner in SQL, plus JSONB gives me the document flexibility anyway. Supabase would also hand me auth + row-level security cheaply.

**I:** Cross-question: your grammar checker misses most errors. Why keep it?

**C:** It's high-precision on common ESL patterns with zero latency and cost, which is what instant feedback needs. I traded recall for determinism deliberately; the LLM path covers depth. It's the right tool for 'immediate, free, offline'.

**I:** You call it "AI-powered" but scoring isn't AI.

**C:** Correct, and I wouldn't overclaim. Scoring is heuristics — cheap, offline, testable. The AI is an optional enhancer for prose. I'd describe it exactly that way on a resume.

**I:** Biggest weakness of the whole thing?

**C:** The client-side OpenAI key when AI is enabled — it's exposed in the browser. The fix is the proxy I described. After that: no tests/CI, and the ~610 kB bundle from Recharts, which I'd lazy-load.

**I:** Tell me about working under ambiguity here.

**C:** The brief was a huge feature wishlist. I scoped an honest MVP: a working deterministic core plus clearly-labelled estimates, rather than faking audio analysis. I documented what's real versus aspirational so expectations stayed grounded — and I fixed the integrity bug the moment testing exposed it.

**I:** Good — you know your code and your trade-offs, and you don't oversell. That's what I look for.

## 22

# Cheat Sheet & Top 100 Questions

## 22.1 Architecture in one breath

Client-only React 18 + TS + Vite SPA · screen state-machine in `App.tsx` · pure domain layer in `lib/` · persistence via localStorage repository · optional browser OpenAI enhancement · no backend/DB/auth/tests/CI.

## 22.2 Files you must be able to name

| File                                       | One-liner                                                                                     |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>App.tsx</code>                       | State machine + session orchestration ( <code>handleFinish</code> )                           |
| <code>lib/analysis/engine.ts</code>        | <code>analyzeSpeech()</code> □ <code>CoachReport</code> ; <code>MIN_SPEECH_WORDS</code> guard |
| <code>lib/analysis/grammar.ts</code>       | 10 regex rules (Strategy)                                                                     |
| <code>lib/analysis/text.ts</code>          | words/sentences/syllables/clamp/countBy                                                       |
| <code>lib/topics.ts</code>                 | 40 seeds × 4 difficulty frames                                                                |
| <code>lib/gamification.ts</code>           | XP=level×100, streaks, 11 badges, insights                                                    |
| <code>lib/storage.ts</code>                | Repository over 3 localStorage keys                                                           |
| <code>lib/llm.ts</code>                    | Optional OpenAI call + graceful fallback                                                      |
| <code>hooks/useSpeechRecognition.ts</code> | Web Speech API wrapper (restart-on-onend)                                                     |
| <code>types.ts</code>                      | All domain types + <code>DURATION_OPTIONS</code>                                              |

## 22.3 "APIs" & "tables"

- **APIs consumed:** Web Speech API (local STT); OpenAI Chat Completions (optional, `gpt-4o-mini`, JSON mode).
- **"Tables" (localStorage):** `speakx.sessions.v1` (Session[]), `speakx.gamification.v1` (GamificationState), `speakx.openai.key.v1` (string).

## 22.4 Numbers to remember

| Metric               | Value                                |
|----------------------|--------------------------------------|
| Runtime dependencies | 3 (react, react-dom, recharts)       |
| Grammar rules        | 10 (regex)                           |
| Score dimensions     | 13                                   |
| Min words to score   | 12 ( <code>MIN_SPEECH_WORDS</code> ) |
| Ideal WPM (scoring)  | 130                                  |
| XP to next level     | level × 100                          |
| Badges               | 11                                   |
| JS bundle            | ~610 kB raw / ~175 kB gzip           |

| Metric            | Value                 |
|-------------------|-----------------------|
| Engine complexity | O(n) time, O(w) space |

## 22.5 Common mistakes to avoid in the interview

- Claiming a backend/DB/auth/tests that don't exist.
- Saying the scoring is "machine learning" — it's heuristics.
- Proposing to hide the key in a client-side `.env` (still public).
- Saying Redux/microservices/queues are needed — they're not, and saying so signals over-engineering.
- Forgetting to volunteer trade-offs when defending a decision.

## 22.6 Top 100 Questions to Revise Before the Interview

Condensed prompts — expand each using the model answers in §18. Grouped for a fast final pass.

1. What does the app do in 30 seconds?
2. Deterministic engine vs. optional LLM — the split?
3. Why client-only? Trade-offs?
4. The three architectural layers?
5. Import direction and why it matters?
6. Why no backend, and when would you add one?
7. Sketch the v2 backend.
8. How does navigation work (state machine)?
9. Why no Redux/Context?
10. Explain `useSpeechRecognition`.
11. Why `useRef` for the recognizer?
12. Chrome auto-stop problem & fix?
13. Stale-closure avoidance in the timer?
14. What re-renders on a tick?
15. Why StrictMode matters here?
16. How would you code-split?
17. Error Boundary — risk & fix?
18. `DURATION_OPTIONS` / `Duration` type trick?
19. Where are discriminated unions used?
20. Which strict flags & why?
21. How is the optional LLM field typed?
22. How is WPM computed + edge case?
23. Filler detection (Set vs regex)?
24. Vocabulary diversity (TTR) + its bias?
25. Grammar checker design; regex vs parser?
26. How are duplicate grammar issues avoided?
27. Structure detection + its weakness?
28. Confidence formula; the silence bug?
29. How does difficulty change scores?
30. What's the `void` dead code?
31. Data structures in the engine & why?
32. Time/space complexity of `analyzeSpeech`?
33. Level-up loop complexity?
34. Syllable heuristic — exact?
35. How is data persisted/modelled?
36. `localStorage` risks?
37. Non-atomic multi-key writes — impact?
38. Postgres schema for v2?
39. Why `handleFinish` is async?
40. Keeping UI responsive during analysis?
41. LLM integration — why optional?
42. Reliable structured output — how?
43. Preventing LLM from duplicating metrics?
44. Prompt injection risk & mitigations?
45. Token/cost estimate per session?
46. Biggest security issue + fix?
47. XSS exposure here?
48. Why CSRF/SQLi are N/A?
49. Scale to 1M users?
50. Where does Redis help?
51. Do you need microservices/queues?
52. Dockerise or not?
53. CAP choice for sync?
54. DB selection reasoning?
55. Current bottlenecks?
56. How is it tested (honestly)?
57. First 5 unit tests?
58. Git workflow?
59. CI/CD you'd add?
60. Functional `setState` — why?
61. `??` vs `||`?

62. `as const` — where critical?
63. A bug you fixed (STAR)?
64. A trade-off you'd defend?
65. Quality without QA?
66. Disagree-and-commit on Redux?
67. "AI-powered" but not AI — respond.
68. Grammar checker "useless" — rebut.
69. Two tabs at once — what happens?
70. Non-English speech — what happens?
71. localStorage full/disabled?
72. Trust a score without audio?
73. Remove `text.ts` — blast radius?
74. Why 600 kB bundle?
75. Critique your own architecture (Principal).
76. Why embed topic+report in a session?
77. Denormalisation trade-offs?
78. How does the streak reset logic work?
79. How are badges unlocked/deduped?
80. How are cross-session insights computed?
81. Why newest-first ( `unshift` ) storage?
82. How does the dashboard build trends?
83. Why Recharts; alternatives?
84. Why Vite over CRA/Webpack?
85. How does `safeParse` protect reads?
86. What happens on OpenAI 429?
87. Why JSON mode isn't enough alone?
88. How would you add i18n?
89. How to move analysis to a Web Worker?
90. How to prune old sessions?
91. How to add a schema version/migration?
92. How to make `awardForSession` pure?
93. Which patterns are present vs absent?
94. Facade — where?
95. Adapter — where (two places)?
96. Strategy — where (two places)?
97. Repository — where?
98. What's your #1 refactor?
99. What would you build with one more week?
100. How do you measure "improvement" over time?
101. What makes this honest/trustworthy to users?
102. If this had 100k paying users, top 3 priorities?

---

End of handbook · Every claim herein is grounded in the source of `RRNauhwar/GD-IMPROVEMENT-APP`; absent capabilities are marked **NOT PRESENT**. Scores: Architecture 7.5 · Scalability 7.0 · Security 5.5 · Maintainability 7.5 · Code Quality 7.5 · Interview-Readiness 9.0 (□ 7.3 overall as an MVP).